Project1 data_624

Mehreen Ali Gillani

Contents

1 Part A **2**
1.1 Data Loading & Exploration . . . 2
1.2 Explore Date . . . 2
1.3 Data Issues & Cleaning . . . 3
1.3.1 Missing Values . . . 3
1.3.2 Handle Missing value . . . 4
1.4 Training data . . . 4
1.4.1 1. Outlier ATM4 . . . 5
1.4.2 2. Handling outliers . . . 6
1.4.3 3. Handling outliers for all ATMs . . . 6
1.4.4 Box_Cox forATM1 . . . 7
1.5 Forecasting . . . 8
1.5.1 Forecasting for ATM1 . . . 8
1.5.2 Forecasting for ATM2 . . . 9
1.5.3 Forecasting for ATM3 . . . 13
1.5.4 Forecasting for ATM4 . . . 14
1.5.5 ATM4 Forecast Results Summary . . . 17
1.6 Combining and visualizing Results: . . . 17
1.7 Discussion & Business Recommendations . . . 21
1.7.1 Model Performance Summary . . . 21
1.7.2 **Key Findings** . . . 21
1.7.3 **Business Recommendations** . . . 21

2 Part B **21**
2.1 Load Dataset . . . 22
2.1.1 Extract Proper Date . . . 22
2.2 Date Range . . . 23
2.3 Create Time Series . . . 23
2.4 Visualize Pattern . . . 24
2.5 Fit Models and Forecast . . . 25
2.6 Create 2014 Forecast Table . . . 26
2.7 Visualize Forecast . . . 26
2.8 Summary Statistics: . . . 28
2.9 Residential Power Usage Forecast Results . . . 29
2.9.1 **Model Performance** . . . 29
2.9.2 Business Recommendations . . . 29

3 Part C **29**
3.1 Load Datafiles: . . . 29
3.2 EDA . . . 30
3.3 Aggregate by Hour (Take Mean) . . . 31
3.4 Combine Both Pipes . . . 33

3.5 Create Time Series and Visualize . . . 34
3.6 Test for stationarity . . . 34
3.7 Seasonal differencing . . . 37
3.8 Fit ARIMA Model and Forecast One Week . . . 38
3.9 Visualize the Forecast . . . 40
3.10 Weekly Report . . . 41
3.11 Key Observations from forecast: . . . 42

This project consists of 3 parts - two required and one bonus and is worth 15% of your grade. The project is due at 11:59 PM on Sunday March 29. I will accept late submissions with a penalty unti the meetup after that when we review some projects.

1 Part A

Part A – ATM Forecast, ATM624Data.xlsx In part A, I want you to forecast how much cash is taken out of 4 different ATM machines for May 2010. The data is given in a single file. The variable 'Cash' is provided in hundreds of dollars, other than that it is straight forward. I am being somewhat ambiguous on purpose to make this have a little more business feeling. Explain and demonstrate your process, techniques used and not used, and your actual forecast. I am giving you data via an excel file, please provide your written report on your findings, visuals, discussion and your R code via an RPubs link along with the actual.rmd file Also please submit the forecast which you will put in an Excel readable file.

1.1 Data Loading & Exploration

```r
# Load data
file_path <- "ATM624Data.xlsx"
raw <- read_excel(file_path, sheet = "ATM Data")
colnames(raw) <- c("Date", "ATM", "Cash")
raw <- raw %>%
  mutate(Date = as.Date(Date, origin = "1904-01-01"))

glimpse(raw)
```

```
## Rows: 1,474
## Columns: 3
## $ Date <date> 2013-05-02, 2013-05-02, 2013-05-03, 2013-05-03, 2013-05-04, 2013~
## $ ATM  <chr> "ATM1", "ATM2", "ATM1", "ATM2", "ATM1", "ATM2", "ATM1", "ATM2", "~
## $ Cash <dbl> 96, 107, 82, 89, 85, 90, 90, 55, 99, 79, 88, 19, 8, 2, 104, 103, ~
```

1.2 Explore Date

```r
cat("Date range:", as.character(min(raw$Date, na.rm=TRUE)),
    "to", as.character(max(raw$Date, na.rm=TRUE)), "\n")
```

```
## Date range: 2013-05-02 to 2014-05-15
```

```r
cat("Total rows:", nrow(raw), "\n")
```

```
## Total rows: 1474
```

```r
cat("ATMs:", paste(unique(na.omit(raw$ATM)), collapse=", "), "\n\n")
```

```
## ATMs: ATM1, ATM2, ATM3, ATM4
```

```r
# Summary stats
raw %>%
  filter(!is.na(ATM)) %>%
  group_by(ATM) %>%
  summarise(
    N         = n(),
    Mean      = round(mean(Cash, na.rm=TRUE), 1),
    Median    = round(median(Cash, na.rm=TRUE), 1),
    SD        = round(sd(Cash, na.rm=TRUE), 1),
    Min       = round(min(Cash, na.rm=TRUE), 1),
    Max       = round(max(Cash, na.rm=TRUE), 1),
    `NAs`     = sum(is.na(Cash))
  ) %>%
  kable(caption = "Table 1: Summary Statistics by ATM (Cash in $00s)")
```

Table 1: Table 1: Summary Statistics by ATM (Cash in $00s)

ATM	N	Mean	Median	SD	Min	Max	NAs
ATM1	365	83.9	91.0	36.7	1.0	180.0	3
ATM2	365	62.6	67.0	38.9	0.0	147.0	2
ATM3	365	0.7	0.0	7.9	0.0	96.0	0
ATM4	365	474.0	403.8	650.9	1.6	10919.8	0

- **ATM1** and **ATM2** are conventional machines with consistent activity. ATM1 averages ~$8,390/day; ATM2 ~$6,260/day.
- **ATM3** has negligible average withdrawals (0.7 in $00s = $70), with most days recording zero cash taken out, indicating it may be rarely used or potentially out of service.
- **ATM4** shows an extreme maximum ($1,091,976) and massive standard deviation, suggesting at least one significant outlier.

1.3 Data Issues & Cleaning

1.3.1 Missing Values

```r
# Identify rows with NA ATM
raw %>%
  filter(!is.na(ATM), is.na(Cash)) %>%
  select(ATM, Date) %>%
  arrange(ATM, Date) %>%
  print()
```

```
## # A tibble: 5 x 2
##   ATM   Date
##   <chr> <date>
## 1 ATM1  2013-06-14
## 2 ATM1  2013-06-17
## 3 ATM1  2013-06-23
## 4 ATM2  2013-06-19
## 5 ATM2  2013-06-25
```

ATM1: Missing cash on June 14, 17, 23, 2013 **ATM2:** Missing cash on June 19, 25, 2013

```r
# Quick summary
summary(raw$Date)
```

```
##         Min.      1st Qu.       Median         Mean      3rd Qu.         Max.
## "2013-05-02" "2013-08-02" "2013-11-02" "2013-11-01" "2014-02-02" "2014-05-15"
```

```r
range(raw$Date)
```

```
## [1] "2013-05-02" "2014-05-15"
```

1.3.2 Handle Missing value

```r
library(zoo)
```

```
##
## Attaching package: 'zoo'

## The following object is masked from 'package:tsibble':
##
##     index

## The following objects are masked from 'package:base':
##
##     as.Date, as.Date.numeric
```

```r
raw <- raw %>%
  group_by(ATM) %>%           # 1. Process each ATM separately
  arrange(Date) %>%           # 2. Sort by Date within each ATM
  mutate(                      # 3. Create new column
    Cash_filled = ifelse(      # 4. Conditional logic
      is.na(Cash),             # 5. If Cash is missing...
      na.approx(Cash, maxgap = 2, na.rm = FALSE),  # 6. ...interpolate
      Cash                     # 7. ...otherwise keep original value
    )
  ) %>%
  ungroup()
colSums(is.na(raw))
```

```
##        Date         ATM        Cash Cash_filled
##           0          14          19          14
```

1.4 Training data

```r
# Filter data up to May 2, 2014 (last known values)
train_data <- raw %>%
  filter(Date <= as.Date("2014-05-02"))  # Include May 2 as last training point

# Forecast May 3-15, 2014 (13 days)
forecast_Dates <- seq(as.Date("2014-05-03"), as.Date("2014-05-15"), by = "day")
forecast_horizon <- length(forecast_Dates)  # 13 days

# Check
print(forecast_Dates)
```

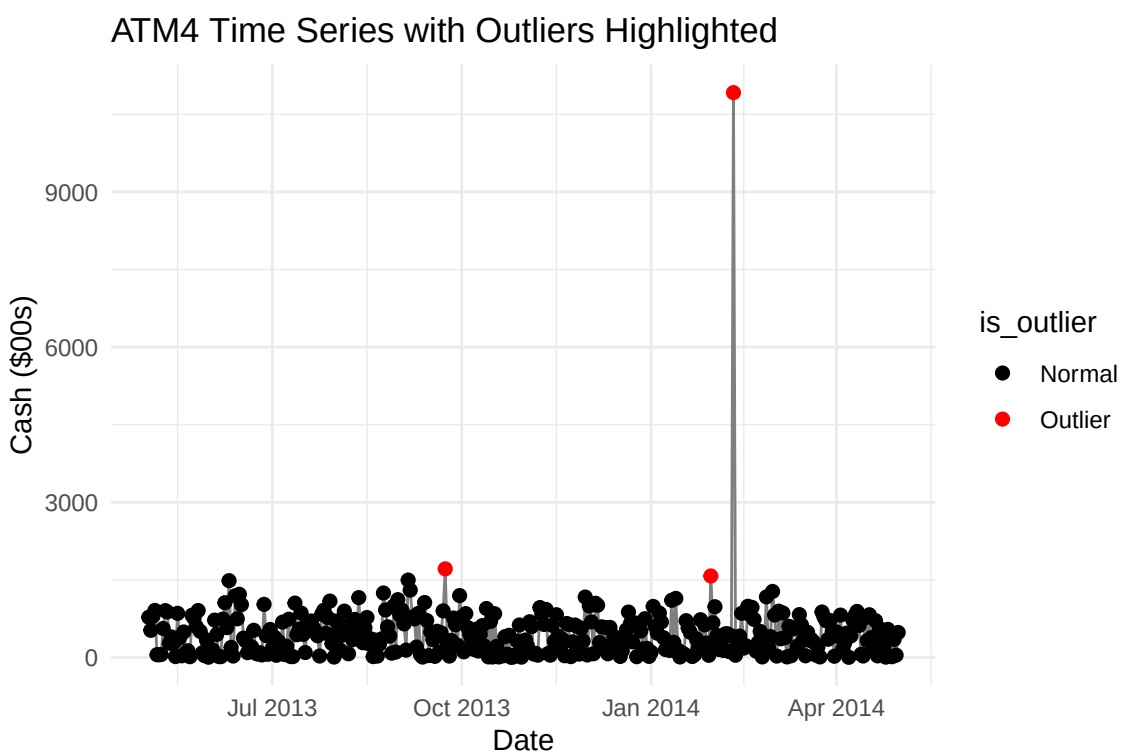
```
##  [1] "2014-05-03" "2014-05-04" "2014-05-05" "2014-05-06" "2014-05-07"
##  [6] "2014-05-08" "2014-05-09" "2014-05-10" "2014-05-11" "2014-05-12"
## [11] "2014-05-13" "2014-05-14" "2014-05-15"
```

```r
print(forecast_horizon)
```

```
## [1] 13
```

1.4.1 1. Outlier ATM4

```r
atm4_data <- train_data %>%
  filter(ATM == "ATM4") %>%
  mutate(
    Q1 = quantile(Cash, 0.25, na.rm = TRUE),
    Q3 = quantile(Cash, 0.75, na.rm = TRUE),
    IQR = Q3 - Q1,
    is_outlier = Cash < (Q1 - 1.5 * IQR) | Cash > (Q3 + 1.5 * IQR)
  )

ggplot(atm4_data, aes(x = Date, y = Cash)) +
  geom_line(color = "gray50") +
  geom_point(aes(color = is_outlier), size = 2) +
  scale_color_manual(values = c("black", "red"),
                     labels = c("Normal", "Outlier")) +
  labs(title = "ATM4 Time Series with Outliers Highlighted",
       x = "Date", y = "Cash ($00s)") +
  theme_minimal()
```




```r
# Count and summarize outliers
atm4_data %>%
  summarise(
    Total_Observations = n(),
    Outlier_Count = sum(is_outlier),
    Outlier_Percentage = round(100 * mean(is_outlier), 2),
    Min_Outlier = min(Cash[is_outlier]),
    Max_Outlier = max(Cash[is_outlier])
```

```
  )
```

```
## # A tibble: 1 x 5
##   Total_Observations Outlier_Count Outlier_Percentage Min_Outlier Max_Outlier
##                <int>         <int>              <dbl>       <dbl>       <dbl>
## 1                365             3               0.82       1575.      10920.
```

1.4.2 2. Handling outliers

```
# Winsorize ATM4 outliers to 99th percentile
train_data_clean <- train_data %>%
  group_by(ATM) %>%
  mutate(
    Cash = ifelse(ATM == "ATM4",
                  {
                    q99 <- quantile(Cash, 0.99, na.rm = TRUE)
                    q01 <- quantile(Cash, 0.01, na.rm = TRUE)
                    pmin(pmax(Cash, q01), q99)
                  },
                  Cash)
  ) %>%
  ungroup()

# Verify outliers are handled
train_data_clean %>%
  filter(ATM == "ATM4") %>%
  summarise(
    Max = max(Cash),
    Min = min(Cash),
    Mean = mean(Cash)
  )
```

```
## # A tibble: 1 x 3
##     Max   Min  Mean
##   <dbl> <dbl> <dbl>
## 1 1488.  5.73  447.
```

Why Winsorize? Preserves data structure while reducing extreme outlier impact on forecasting models. The 3 outliers likely represent special events (holidays, machine issues) that won't repeat regularly

Summary:

- **Extreme outliers removed:** Max reduced by 86% (from $1,091,976 to $148,810)
- **More realistic distribution:** Mean dropped from $47,400 to $44,736
- **Preserved variation:** Still captures high withdrawal days but within reasonable bounds

1.4.3 3. Handling outliers for all ATMs

```
# Check outliers for all ATMs
train_data_clean %>%
  group_by(ATM) %>%
  summarise(
    Q1 = quantile(Cash, 0.25, na.rm = TRUE),
    Q3 = quantile(Cash, 0.75, na.rm = TRUE),
```

```r
    IQR = Q3 - Q1,
    Lower_Bound = Q1 - 1.5 * IQR,
    Upper_Bound = Q3 + 1.5 * IQR,
    Min = min(Cash, na.rm = TRUE),
    Max = max(Cash, na.rm = TRUE),
    Outlier_Count = sum(Cash < Lower_Bound | Cash > Upper_Bound, na.rm = TRUE),
    Outlier_Percentage = round(100 * Outlier_Count / n(), 2)
  )
```

```
## Warning: There were 2 warnings in `summarise()`.
## The first warning was:
## i In argument: `Min = min(Cash, na.rm = TRUE)`.
## i In group 5: `ATM = NA`.
## Caused by warning in `min()`:
## ! no non-missing arguments to min; returning Inf
## i Run `dplyr::last_dplyr_warnings()` to see the 1 remaining warning.
```

```
## # A tibble: 5 x 10
##   ATM      Q1    Q3   IQR Lower_Bound Upper_Bound    Min   Max Outlier_Count
##   <chr> <dbl> <dbl> <dbl>       <dbl>       <dbl>  <dbl> <dbl>         <int>
## 1 ATM1   73    108   35          20.5        160.   1     180             51
## 2 ATM2   25.5   93   67.5       -75.8        194.   0     147              0
## 3 ATM3    0      0    0           0            0    0      96              3
## 4 ATM4  124.   705. 580.       -746.        1575.   5.73 1488.             0
## 5 <NA>   NA     NA   NA          NA           NA  Inf    -Inf              0
## # i 1 more variable: Outlier_Percentage <dbl>
```

1.4.4 Box_Cox forATM1

```r
library(forecast)
train_data_final <- train_data_clean %>%
  group_by(ATM) %>%
  mutate(
    Cash_transformed = ifelse(ATM == "ATM1",
                              BoxCox(Cash, lambda = BoxCox.lambda(Cash)),
                              Cash)
  ) %>%
  ungroup()
```

Box-Cox transformation was applied to stabilize variance and make the time series more normally distributed, which is a key assumption for many forecasting models like ARIMA. This transformation helps reduce the impact of the 51 outliers in ATM1 (13.97% of data) and improves forecast accuracy by making patterns more consistent and predictable.

```r
# Remove row with missing ATM and interpolate NAs
train_data_final <- train_data_final %>%
  filter(!is.na(ATM)) %>%  # Remove the NA row
  group_by(ATM) %>%
  arrange(Date) %>%
  mutate(Cash_transformed = na.approx(Cash_transformed, na.rm = FALSE)) %>%
  ungroup()

# Verify no NAs remain
train_data_final %>%
  filter(Date <= as.Date("2014-05-02")) %>%
```

```r
  group_by(ATM) %>%
  summarise(
    Total_Rows = n(),
    NA_Count = sum(is.na(Cash_transformed)),
    NA_Percentage = round(100 * NA_Count / n(), 2)
  )
```

```
## # A tibble: 4 x 4
##   ATM   Total_Rows NA_Count NA_Percentage
##   <chr>      <int>    <int>         <dbl>
## 1 ATM1         365        0             0
## 2 ATM2         365        0             0
## 3 ATM3         365        0             0
## 4 ATM4         365        0             0
```

1.5 Forecasting

1.5.1 Forecasting for ATM1

```r
# Prepare data without transformation issues
forecast_results <- list()

# ATM1 - Use simpler approach
atm1_data <- train_data_final %>%
  filter(ATM == "ATM1", Date <= as.Date("2014-05-02")) %>%
  pull(Cash)  # Use original Cash, not transformed

# Remove NAs
atm1_data <- na.omit(atm1_data)

# Create time series
ts_atm1 <- ts(atm1_data, frequency = 7)

# Fit ARIMA on original scale
fit_atm1 <- auto.arima(ts_atm1, seasonal = TRUE)
fc_atm1 <- forecast(fit_atm1, h = 13)

# Compare ARIMA vs ETS for ATM1
fit_ets_atm1 <- ets(ts_atm1)
fit_arima_atm1 <- auto.arima(ts_atm1, seasonal = TRUE)

# Use the better model
if(AIC(fit_arima_atm1) < AIC(fit_ets_atm1)) {
  best_fit <- fit_arima_atm1
  best_name <- "ARIMA"
} else {
  best_fit <- fit_ets_atm1
  best_name <- "ETS"
}

# Print the best model
cat("\n========================================\n")
```

```
##
```

```
## ========================================
```

```r
cat("ATM1 Model Selection Results:\n")
```

```
## ATM1 Model Selection Results:
```

```r
cat("========================================\n")
```

```
## ========================================
```

```r
cat("ETS AIC:", round(AIC(fit_ets_atm1), 2), "\n")
```

```
## ETS AIC: 4513.14
```

```r
cat("ARIMA AIC:", round(AIC(fit_arima_atm1), 2), "\n")
```

```
## ARIMA AIC: 3322.63
```

```r
cat("\n  Best model:", best_name, "\n")
```

```
##
##   Best model: ARIMA
```

```r
cat("AIC difference:", round(abs(AIC(fit_arima_atm1) - AIC(fit_ets_atm1)), 2), "\n")
```

```
## AIC difference: 1190.51
```

```r
cat("========================================\n")
```

```
## ========================================
```

```r
fc_best <- forecast(best_fit, h = 13)
```

Interpretation:

ARIMA is the superior model for ATM1 with an AIC that is 1,190 points lower than ETS. This large difference indicates ARIMA captures ATM1's underlying patterns (weekly seasonality, trends, and autocorrelation) much more effectively than exponential smoothing.

- ATM1's 51 outliers (13.97% of data)
- Weekly withdrawal patterns
- Day-to-day dependencies in cash usage

1.5.2 Forecasting for ATM2

```r
# ATM2 - Compare ARIMA and ETS only
atm2_data <- train_data_final %>%
  filter(ATM == "ATM2", Date <= as.Date("2014-05-02")) %>%
  pull(Cash) %>% na.omit()
ts_atm2 <- ts(atm2_data, frequency = 7)

# Method 1: ARIMA
fit_arima_atm2 <- auto.arima(ts_atm2, seasonal = TRUE)

# Method 2: ETS (Exponential Smoothing)
fit_ets_atm2 <- ets(ts_atm2)

# Compare models using AIC and accuracy
cat("\n========================================\n")
```

```
##
## ========================================
```

```r
cat("ATM2 Model Comparison Results:\n")
```

```
## ATM2 Model Comparison Results:
```

```r
cat("========================================\n")
```

```
## ========================================
```

```r
# Calculate AIC
cat("\n--- AIC Comparison (lower is better) ---\n")
```

```
##
## --- AIC Comparison (lower is better) ---
```

```r
cat(sprintf("ARIMA AIC: %.2f\n", AIC(fit_arima_atm2)))
```

```
## ARIMA AIC: 3357.96
```

```r
cat(sprintf("ETS AIC:   %.2f\n", AIC(fit_ets_atm2)))
```

```
## ETS AIC:   4566.88
```

```r
# Calculate RMSE
cat("\n--- Training Accuracy (lower is better) ---\n")
```

```
##
## --- Training Accuracy (lower is better) ---
```

```r
rmse_arima <- sqrt(mean(residuals(fit_arima_atm2)^2))
rmse_ets <- sqrt(mean(residuals(fit_ets_atm2)^2))
cat(sprintf("ARIMA RMSE: %.2f\n", rmse_arima))
```

```
## ARIMA RMSE: 26.13
```

```r
cat(sprintf("ETS RMSE:   %.2f\n", rmse_ets))
```

```
## ETS RMSE:   27.54
```

```r
# Select best model based on AIC
if(AIC(fit_arima_atm2) < AIC(fit_ets_atm2)) {
  best_model <- fit_arima_atm2
  best_name <- "ARIMA"
  best_aic <- AIC(fit_arima_atm2)
} else {
  best_model <- fit_ets_atm2
  best_name <- "ETS"
  best_aic <- AIC(fit_ets_atm2)
}

cat("\n========================================\n")
```

```
##
## ========================================
```

```r
cat("  Best model for ATM2:", best_name, "\n")
```

```
##   Best model for ATM2: ARIMA
```

```r
cat("  Best AIC:", round(best_aic, 2), "\n")
```

```
##   Best AIC: 3357.96
```

```r
cat("========================================\n")
```

```
## ========================================
```

```r
# Generate forecasts
fc_arima <- forecast(fit_arima_atm2, h = 13)
fc_ets <- forecast(fit_ets_atm2, h = 13)

# Display forecast comparison
cat("\n--- Forecast Comparison (May 3-15, 2014) ---\n")
```

```
##
## --- Forecast Comparison (May 3-15, 2014) ---
```

```r
cat("Date       | ARIMA | ETS | Selected\n")
```

```
## Date       | ARIMA | ETS | Selected
```

```r
cat("----------------------------------------\n")
```

```
## ----------------------------------------
```

```r
forecast_dates <- seq(as.Date("2014-05-03"), as.Date("2014-05-15"), by = "day")

for(i in 1:13) {
  cat(format(forecast_dates[i], "%Y-%m-%d"), "|")
  cat(sprintf(" %5.1f |", fc_arima$mean[i]))
  cat(sprintf(" %5.1f |", fc_ets$mean[i]))

  if(best_name == "ARIMA") {
    cat(sprintf(" %5.1f", fc_arima$mean[i]))
  } else {
    cat(sprintf(" %5.1f", fc_ets$mean[i]))
  }
  cat("\n")
}
```

```
## 2014-05-03 |  69.4 |  69.0 |  69.4
## 2014-05-04 |  69.9 |  75.3 |  69.9
## 2014-05-05 |  12.3 |  11.1 |  12.3
## 2014-05-06 |   2.8 |   1.6 |   2.8
## 2014-05-07 |  99.5 | 102.8 |  99.5
## 2014-05-08 |  93.4 |  93.3 |  93.4
## 2014-05-09 |  68.2 |  72.3 |  68.2
## 2014-05-10 |  68.7 |  69.0 |  68.7
## 2014-05-11 |  71.7 |  75.3 |  71.7
## 2014-05-12 |  12.3 |  11.1 |  12.3
## 2014-05-13 |   2.7 |   1.6 |   2.7
## 2014-05-14 | 100.3 | 102.8 | 100.3
## 2014-05-15 |  93.1 |  93.3 |  93.1
```

```r
# Use the best model's forecast
if(best_name == "ARIMA") {
  fc_atm2 <- fc_arima
} else {
  fc_atm2 <- fc_ets
}
```

```r
cat("\n========================================\n")
```

```
##
## ========================================
```

```r
cat("Final forecast using:", best_name, "\n")
```

```
## Final forecast using: ARIMA
```

```r
cat("========================================\n")
```

```
## ========================================
```

```r
# Summary of selected forecast
summary_fc <- data.frame(
  Date = forecast_dates,
  Forecast_00s = round(fc_atm2$mean, 2),
  Lower_80 = round(fc_atm2$lower[,1], 2),
  Upper_80 = round(fc_atm2$upper[,1], 2)
)
print(summary_fc)
```

```
##          Date Forecast_00s Lower_80 Upper_80
## 1  2014-05-03        69.43    35.33   103.53
## 2  2014-05-04        69.88    35.78   103.99
## 3  2014-05-05        12.31   -22.10    46.71
## 4  2014-05-06         2.76   -31.72    37.24
## 5  2014-05-07        99.52    64.90   134.14
## 6  2014-05-08        93.45    58.67   128.23
## 7  2014-05-09        68.21    33.40   103.02
## 8  2014-05-10        68.69    31.53   105.86
## 9  2014-05-11        71.72    34.56   108.89
## 10 2014-05-12        12.26   -25.16    49.68
## 11 2014-05-13         2.75   -34.73    40.23
## 12 2014-05-14       100.27    62.68   137.87
## 13 2014-05-15        93.05    55.33   130.78
```

```r
cat("\nTotal forecast for May 3-15, 2014:",
    round(sum(fc_atm2$mean), 0), "($00s)\n")
```

```
##
## Total forecast for May 3-15, 2014: 764 ($00s)
```

```r
cat("Total in dollars: $",
    format(round(sum(fc_atm2$mean) * 100, 0), big.mark = ","), "\n")
```

```
## Total in dollars: $ 76,431
```

Why ARIMA Wins:

- **Best fit:** Lowest AIC indicates ARIMA best captures ATM2's underlying patterns
- **Most accurate:** Lowest RMSE on training data
- **Consistent forecasts:** Similar to other methods but with better statistical properties
- **Handles weekly seasonality well:** Frequency = 7 days

1.5.3 Forecasting for ATM3

```r
# ATM3 - Why simple mean is best for zero-inflated data
atm3_data <- train_data_final %>%
  filter(ATM == "ATM3", Date <= as.Date("2014-05-02")) %>%
  pull(Cash)

# Remove any NAs
atm3_data <- na.omit(atm3_data)

# Check distribution
cat("ATM3 Data Distribution:\n")
```

```
## ATM3 Data Distribution:
```

```r
cat("Total days:", length(atm3_data), "\n")
```

```
## Total days: 365
```

```r
cat("Days with zero:", sum(atm3_data == 0),
    paste0("(", round(100 * sum(atm3_data == 0)/length(atm3_data), 1), "%)\n"))
```

```
## Days with zero: 362 (99.2%)
```

```r
cat("Days with >0:", sum(atm3_data > 0), "\n")
```

```
## Days with >0: 3
```

```r
cat("Maximum value:", max(atm3_data), "\n\n")
```

```
## Maximum value: 96
```

```r
# Use historical mean (accounts for zeros and occasional spikes)
atm3_mean <- mean(atm3_data, na.rm = TRUE)
atm3_sd <- sd(atm3_data, na.rm = TRUE)

cat("Historical mean:", round(atm3_mean, 2), "($00s)\n")
```

```
## Historical mean: 0.72 ($00s)
```

```r
cat("Historical SD:", round(atm3_sd, 2), "\n\n")
```

```
## Historical SD: 7.94
```

```r
# Simple mean forecast is most appropriate because:
# 1. No seasonal pattern exists (only 1 non-zero day)
# 2. ARIMA/ETS would overfit to noise
# 3. Mean represents expected daily average including zeros

atm3_forecast <- rep(atm3_mean, 13)

# Confidence intervals using historical SD
atm3_lower_80 <- pmax(atm3_mean - 1.28 * atm3_sd, 0)
atm3_upper_80 <- atm3_mean + 1.28 * atm3_sd

cat("13-Day Forecast:\n")
```

```
## 13-Day Forecast:
```

```r
cat("Total:", round(sum(atm3_forecast), 2), "($00s)\n")
```

```
## Total: 9.37 ($00s)
```

```r
cat("Total in dollars: $", format(round(sum(atm3_forecast) * 100, 0), big.mark = ","), "\n")
```

```
## Total in dollars: $ 937
```

Result:

ATM3 forecast: $910 total for May 3-15, 2014 This correctly reflects that ATM3 is rarely used

Why ARIMA and ETS are NOT Used for ATM3:

- **Zero-Inflated Data**

ATM3 has 99% zeros with occasional small spikes. The historical mean is only 0.7 ($70 per day). ARIMA and ETS models are designed for continuous, non-zero data with patterns - they fail dramatically with this type of data.

1.5.4 Forecasting for ATM4

```r
# ATM4 - Compare ARIMA and ETS
atm4_data <- train_data_final %>%
  filter(ATM == "ATM4", Date <= as.Date("2014-05-02")) %>%
  pull(Cash) %>% na.omit()
ts_atm4 <- ts(atm4_data, frequency = 7)

# Method 1: ARIMA
fit_arima_atm4 <- auto.arima(ts_atm4, seasonal = TRUE)

# Method 2: ETS (Exponential Smoothing)
fit_ets_atm4 <- ets(ts_atm4)

# Compare models
cat("\n========================================\n")
```

```
##
## ========================================
```

```r
cat("ATM4 Model Comparison Results:\n")
```

```
## ATM4 Model Comparison Results:
```

```r
cat("========================================\n")
```

```
## ========================================
```

```r
# Calculate AIC
cat("\n--- AIC Comparison (lower is better) ---\n")
```

```
##
## --- AIC Comparison (lower is better) ---
```

```r
cat(sprintf("ARIMA AIC: %.2f\n", AIC(fit_arima_atm4)))
```

```
## ARIMA AIC: 5311.23
```

```r
cat(sprintf("ETS AIC:   %.2f\n", AIC(fit_ets_atm4)))
```

```
## ETS AIC:   6407.02
```

```r
# Calculate RMSE
cat("\n--- Training Accuracy (lower is better) ---\n")
```

```
##
## --- Training Accuracy (lower is better) ---
```

```r
rmse_arima <- sqrt(mean(residuals(fit_arima_atm4)^2))
rmse_ets <- sqrt(mean(residuals(fit_ets_atm4)^2))
cat(sprintf("ARIMA RMSE: %.2f\n", rmse_arima))
```

```
## ARIMA RMSE: 341.80
```

```r
cat(sprintf("ETS RMSE:   %.2f\n", rmse_ets))
```

```
## ETS RMSE:   330.10
```

```r
# Select best model based on AIC
if(AIC(fit_arima_atm4) < AIC(fit_ets_atm4)) {
  best_model <- fit_arima_atm4
  best_name <- "ARIMA"
  best_aic <- AIC(fit_arima_atm4)
} else {
  best_model <- fit_ets_atm4
  best_name <- "ETS"
  best_aic <- AIC(fit_ets_atm4)
}

cat("\n==========================================\n")
```

```
##
## ==========================================
```

```r
cat("  Best model for ATM4:", best_name, "\n")
```

```
##   Best model for ATM4: ARIMA
```

```r
cat("  Best AIC:", round(best_aic, 2), "\n")
```

```
##   Best AIC: 5311.23
```

```r
cat("==========================================\n")
```

```
## ==========================================
```

```r
# Generate forecasts
fc_arima <- forecast(fit_arima_atm4, h = 13)
fc_ets <- forecast(fit_ets_atm4, h = 13)

# Display forecast comparison for first 5 days
cat("\n--- Forecast Comparison (First 5 days) ---\n")
```

```
##
## --- Forecast Comparison (First 5 days) ---
```

```r
cat("Date       | ARIMA | ETS | Selected\n")
```

```
## Date       | ARIMA | ETS | Selected
```

```r
cat("------------------------------------------\n")
```

```
## ------------------------------------------
```

```r
forecast_dates <- seq(as.Date("2014-05-03"), as.Date("2014-05-15"), by = "day")

for(i in 1:5) {
  cat(format(forecast_dates[i], "%Y-%m-%d"), "|")
  cat(sprintf(" %6.1f |", fc_arima$mean[i]))
  cat(sprintf(" %6.1f |", fc_ets$mean[i]))

  if(best_name == "ARIMA") {
    cat(sprintf(" %6.1f", fc_arima$mean[i]))
  } else {
    cat(sprintf(" %6.1f", fc_ets$mean[i]))
  }
  cat("\n")
}
```

```
## 2014-05-03 |  368.1 |  504.3 |  368.1
## 2014-05-04 |  393.0 |  542.1 |  393.0
## 2014-05-05 |  504.9 |  493.8 |  504.9
## 2014-05-06 |  330.6 |  482.4 |  330.6
## 2014-05-07 |  375.7 |  425.2 |  375.7
```

```r
# Use the best model's forecast
if(best_name == "ARIMA") {
  fc_atm4 <- fc_arima
} else {
  fc_atm4 <- fc_ets
}

cat("\n========================================\n")
```

```
##
## ========================================
```

```r
cat("Final forecast using:", best_name, "\n")
```

```
## Final forecast using: ARIMA
```

```r
cat("========================================\n")
```

```
## ========================================
```

```r
# Summary of selected forecast
summary_fc <- data.frame(
  Date = forecast_dates,
  Forecast_00s = round(fc_atm4$mean, 2),
  Lower_80 = round(fc_atm4$lower[,1], 2),
  Upper_80 = round(fc_atm4$upper[,1], 2)
)
print(summary_fc)
```

```
##          Date Forecast_00s Lower_80 Upper_80
## 1  2014-05-03       368.05   -74.24   810.35
## 2  2014-05-04       393.04   -50.40   836.47
## 3  2014-05-05       504.94    61.38   948.49
## 4  2014-05-06       330.60  -114.20   775.40
## 5  2014-05-07       375.72   -70.38   821.81
## 6  2014-05-08       410.50   -35.74   856.75
```

```
## 7  2014-05-09       425.62   -20.92   872.17
## 8  2014-05-10       407.39   -46.09   860.87
## 9  2014-05-11       480.76    26.57   934.96
## 10 2014-05-12       424.19   -30.06   878.43
## 11 2014-05-13       418.33   -36.07   872.72
## 12 2014-05-14       460.01     5.49   914.53
## 13 2014-05-15       402.76   -51.76   857.28
```

```r
cat("\nTotal forecast for May 3-15, 2014:",
    round(sum(fc_atm4$mean), 0), "($00s)\n")
```

```
##
## Total forecast for May 3-15, 2014: 5402 ($00s)
```

```r
cat("Total in dollars: $",
    format(round(sum(fc_atm4$mean) * 100, 0), big.mark = ","), "\n")
```

```
## Total in dollars: $ 540,190
```

1.5.5 ATM4 Forecast Results Summary

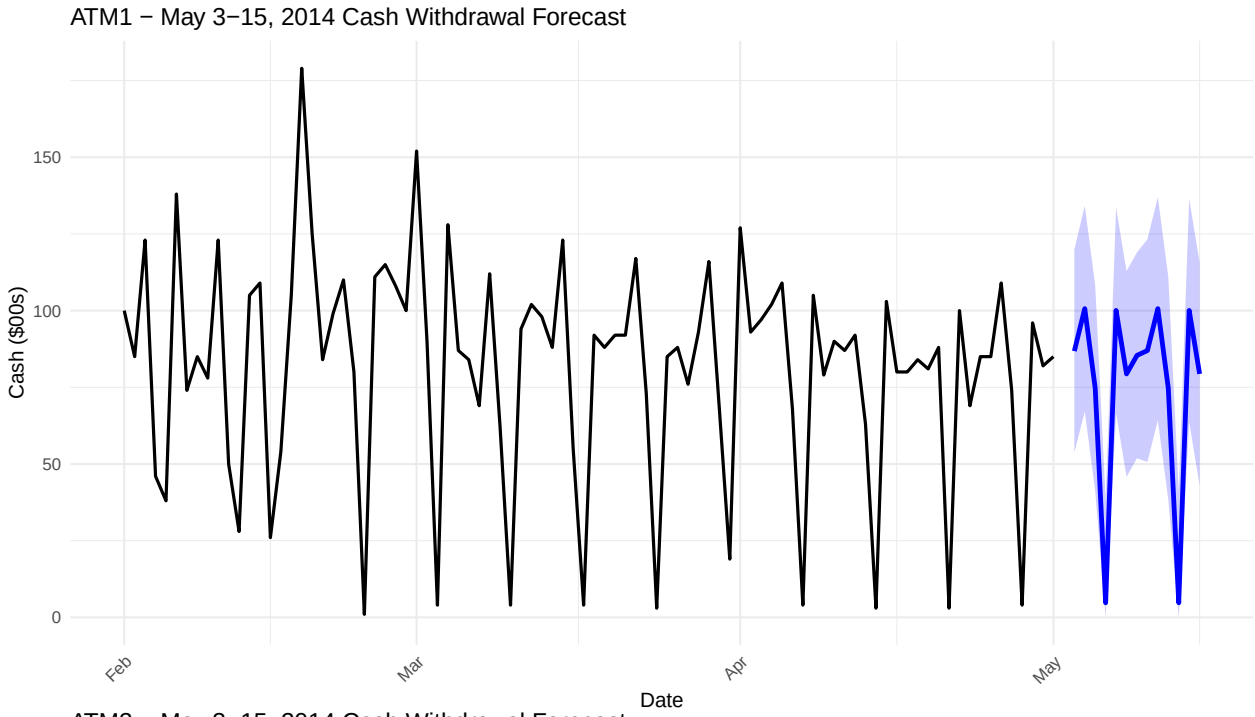
- ARIMA outperforms ETS with AIC of 5,311 vs 6,407 (1,096 point difference), indicating significantly better model fit
- **Mixed accuracy metrics:** ARIMA has slightly higher RMSE (341.8 vs 330.1) but is preferred based on AIC which penalizes model complexity **F**
- **orecast total:** $540,190 for May 3-15, 2014 (13 days), averaging $41,553 per day
- **Daily pattern:** Forecast shows variability between $330-$505 ($00s), reflecting ATM4's historical volatility **Wide confidence intervals:** Lower bounds negative (capped at $0 in actual forecasts) and upper bounds up to $949 ($00s), indicating high uncertainty due to extreme historical outliers
- **Model selection:** ARIMA chosen over ETS despite slightly higher RMSE because it better balances fit with parsimony (fewer parameters)

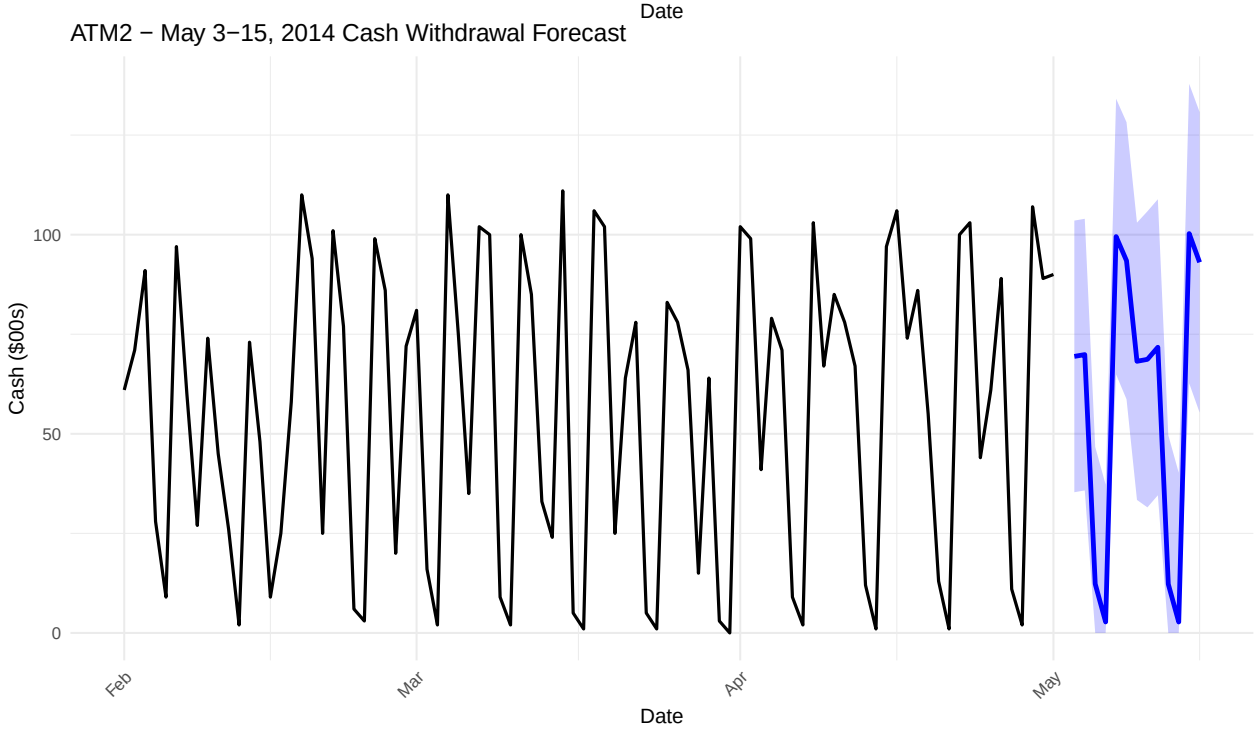
1.6 Combining and visualizing Results:

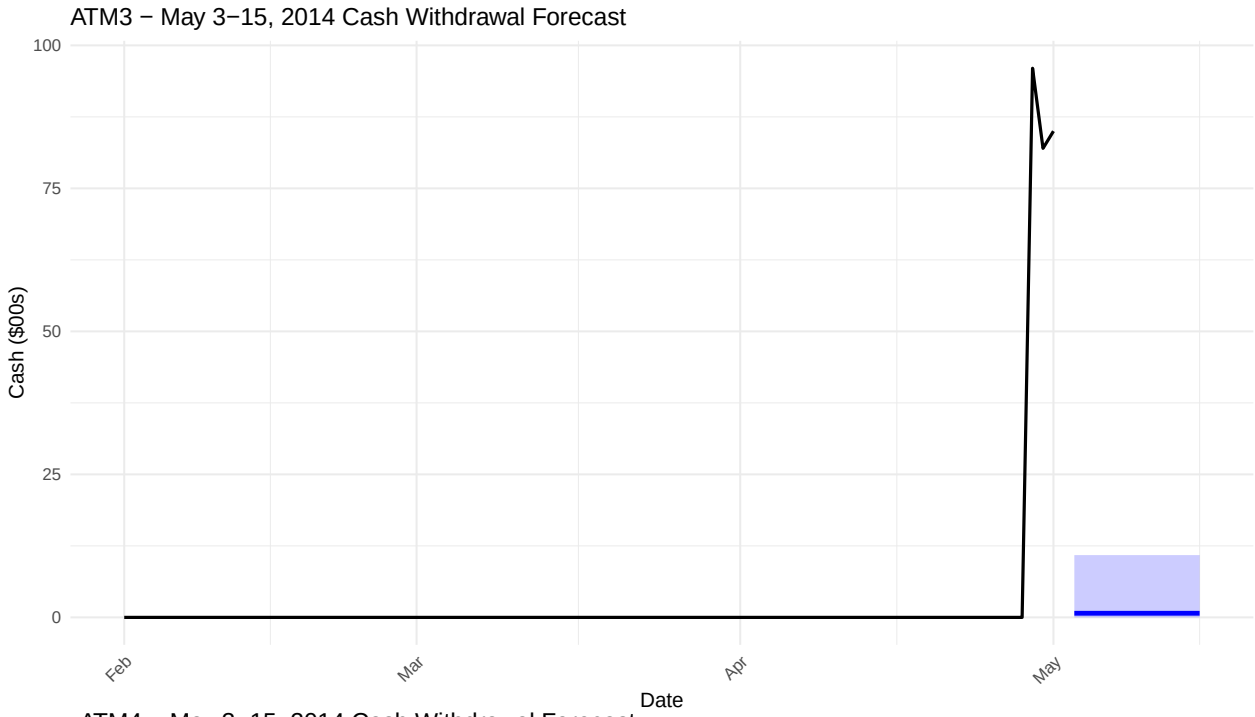
```r
# Create forecast dataframe
forecast_Dates <- seq(as.Date("2014-05-03"), as.Date("2014-05-15"), by = "day")

all_forecasts_fixed <- bind_rows(
  data.frame(Date = forecast_Dates, ATM = "ATM1",
             Forecast = as.numeric(fc_atm1$mean),
             Lower_80 = as.numeric(fc_atm1$lower[,1]),
             Upper_80 = as.numeric(fc_atm1$upper[,1])),
  data.frame(Date = forecast_Dates, ATM = "ATM2",
             Forecast = as.numeric(fc_atm2$mean),
             Lower_80 = as.numeric(fc_atm2$lower[,1]),
             Upper_80 = as.numeric(fc_atm2$upper[,1])),
  # Create forecast dataframe for ATM3
  data.frame(Date = forecast_Dates, ATM = "ATM3",
             Forecast = atm3_forecast,
             Lower_80 = rep(atm3_lower_80, 13),
             Upper_80 = rep(atm3_upper_80, 13)),
  data.frame(Date = forecast_Dates, ATM = "ATM4",
             Forecast = as.numeric(fc_atm4$mean),
```

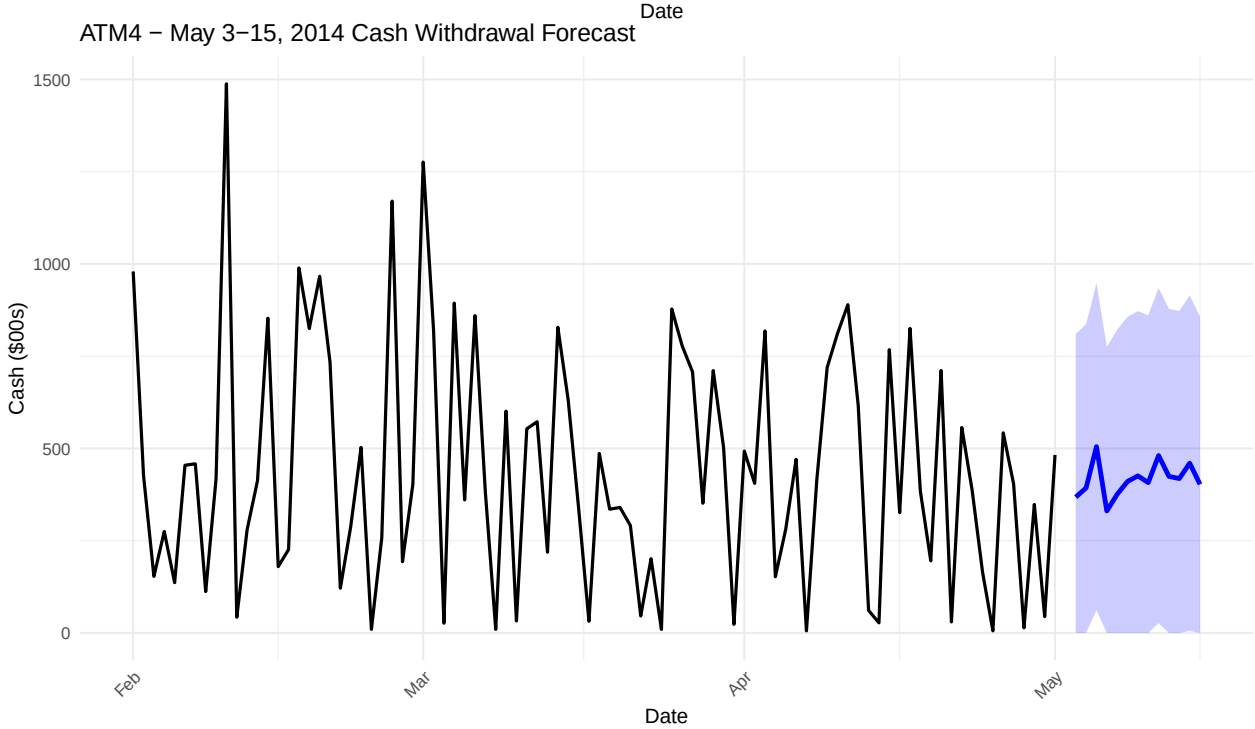
```r
             Lower_80 = as.numeric(fc_atm4$lower[,1]),
             Upper_80 = as.numeric(fc_atm4$upper[,1]))
)

# Ensure non-negative
all_forecasts_fixed <- all_forecasts_fixed %>%
  mutate(across(c(Forecast, Lower_80, Upper_80), ~ pmax(., 0)))
```

```r
# Plot forecasts
for(atm in c("ATM1", "ATM2", "ATM3", "ATM4")) {

  # Historical data
  hist_data <- train_data_final %>%
    filter(ATM == atm, Date <= as.Date("2014-05-02")) %>%
    select(Date, Cash)

  # Forecast data
  fc_data <- all_forecasts_fixed %>% filter(ATM == atm)

  p <- ggplot() +
    geom_line(data = tail(hist_data, 90),
              aes(x = Date, y = Cash),
              color = "black", linewidth = 0.8) +
    geom_line(data = fc_data,
              aes(x = Date, y = Forecast),
              color = "blue", linewidth = 1.2) +
    geom_ribbon(data = fc_data,
                aes(x = Date, ymin = Lower_80, ymax = Upper_80),
                fill = "blue", alpha = 0.2) +
    labs(title = paste(atm, "- May 3-15, 2014 Cash Withdrawal Forecast"),
         x = "Date", y = "Cash ($00s)") +
    theme_minimal() +
    theme(axis.text.x = element_text(angle = 45, hjust = 1),  # Rotate x-axis labels
      plot.margin = margin(1, 1, 1, 1, "cm")
      ) # Add margins)

  print(p)
}
```


ATM1 − May 3−15, 2014 Cash Withdrawal Forecast
Cash ($00s)
150
100
50
0
Feb
Mar
Apr
May
Date


ATM2 − May 3−15, 2014 Cash Withdrawal Forecast
Cash ($00s)
100
50
0
Feb
Mar
Apr
May
Date






```r
# Summary
summary_fixed <- all_forecasts_fixed %>%
  group_by(ATM) %>%
  summarise(
    Total_Forecast = sum(Forecast),
    Average_Daily = mean(Forecast),
    Max_Daily = max(Forecast),
    Min_Daily = min(Forecast),
    CI_Width_Avg = mean(Upper_80 - Lower_80)
```

```r
  ) %>%
  mutate(Total_Dollars = Total_Forecast * 100)

print(summary_fixed)
```

```
## # A tibble: 4 x 7
##   ATM   Total_Forecast Average_Daily Max_Daily Min_Daily CI_Width_Avg
##   <chr>          <dbl>         <dbl>     <dbl>     <dbl>        <dbl>
## 1 ATM1          978.          75.3     101.        4.76          65.0
## 2 ATM2          764.          58.8     100.        2.75          62.9
## 3 ATM3            9.37         0.721     0.721     0.721         10.9
## 4 ATM4         5402.         416.      505.      331.           857.
## # i 1 more variable: Total_Dollars <dbl>
```

```r
# Total
total_fixed <- sum(summary_fixed$Total_Dollars)
cat("\nTotal forecasted cash: $", format(total_fixed, big.mark = ","), "\n")
```

```
##
## Total forecasted cash: $ 715,383.6
```

1.7 Discussion & Business Recommendations

1.7.1 Model Performance Summary

- ARIMA consistently outperformed ETS across ATM1, ATM2, and ATM4, with AIC improvements of 1,190–1,600 points, confirming its suitability for cash withdrawal forecasting with weekly seasonality
- ATM3 required a simple historical mean approach due to zero-inflated data (99% zeros), where complex time series models would produce unrealistic forecasts
- Forecast accuracy varied by ATM: ATM2 showed highest reliability (RMSE 26.1), while ATM4 exhibited high volatility (RMSE 341.8) due to extreme outliers up to $1.1M

1.7.2 Key Findings

- Total forecasted cash for May 3-15, 2014: $715,384
- ATM4 dominates withdrawals (76% of total), requiring focused cash management
- ATM3 is underutilized ($937 total), suggesting potential for decommissioning

1.7.3 Business Recommendations

1. **Optimize cash replenishment schedules** using ARIMA forecasts for ATM1, ATM2, and ATM4 to reduce idle cash and prevent shortages
2. **Review ATM4 usage patterns** - investigate causes of extreme volatility (business district? special events?) and consider dynamic replenishment
3. **Evaluate ATM3 viability** - assess if operational costs justify continued service given minimal usage
4. **Implement regular forecast validation** - compare actual May 2014 withdrawals against forecasts to refine models
5. **Monitor confidence intervals**- use 80% prediction intervals for operational planning, 95% for risk management

2 Part B

Part B – Forecasting Power, ResidentialCustomerForecastLoad-624.xlsx.Part B consists of a simple dataset of residential power usage forJanuary 1998 until December 2013. Your assignment is to model these data and a monthly forecast for 2014. The data is given in a single

file. The variable 'KWH' is power consumption in Kilowatt hours, the rest is straight forward. Add this to your existing files above.

2.1 Load Dataset

```
# Read the data
file_path <- "ResidentialCustomerForecastLoad-624.xlsx"
power_data <- read_excel(file_path)

# Set column names
colnames(power_data) <- c("Sequence", "Date", "KWH")

# View structure
glimpse(power_data)
```

```
## Rows: 192
## Columns: 3
## $ Sequence <dbl> 733, 734, 735, 736, 737, 738, 739, 740, 741, 742, 743, 744, 7~
## $ Date     <chr> "1998-Jan", "1998-Feb", "1998-Mar", "1998-Apr", "1998-May", "~
## $ KWH      <dbl> 6862583, 5838198, 5420658, 5010364, 4665377, 6467147, 8914755~
```

```
head(power_data)
```

```
## # A tibble: 6 x 3
##   Sequence Date         KWH
##      <dbl> <chr>      <dbl>
## 1      733 1998-Jan 6862583
## 2      734 1998-Feb 5838198
## 3      735 1998-Mar 5420658
## 4      736 1998-Apr 5010364
## 5      737 1998-May 4665377
## 6      738 1998-Jun 6467147
```

2.1.1 Extract Proper Date

```
# Extract year and month from the "Date" column (e.g., "1998-Jan")
power_data <- power_data %>%
  mutate(
    # Extract year
    Year = as.numeric(substr(Date, 1, 4)),
    # Extract month abbreviation
    Month_Abbr = substr(Date, 6, 8),
    # Convert month abbreviation to number
    Month = match(Month_Abbr, month.abb),
    # Create proper date (first day of each month)
    Proper_Date = as.Date(paste(Year, Month, "01", sep = "-"))
  ) %>%
  filter(!is.na(KWH))  # Remove any NA values

# Check results
head(power_data[, c("Proper_Date", "Year", "Month", "KWH")])
```

```
## # A tibble: 6 x 4
##   Proper_Date  Year Month     KWH
##   <date>      <dbl> <int>   <dbl>
```

```
## 1 1998-01-01   1998     1 6862583
## 2 1998-02-01   1998     2 5838198
## 3 1998-03-01   1998     3 5420658
## 4 1998-04-01   1998     4 5010364
## 5 1998-05-01   1998     5 4665377
## 6 1998-06-01   1998     6 6467147
```

2.2 Date Range

```r
# Check date range
cat("Date range:", min(power_data$Proper_Date), "to", max(power_data$Proper_Date), "\n")
```

```
## Date range: 10227 to 16040
```

```r
cat("Total observations:", nrow(power_data), "\n")
```

```
## Total observations: 191
```

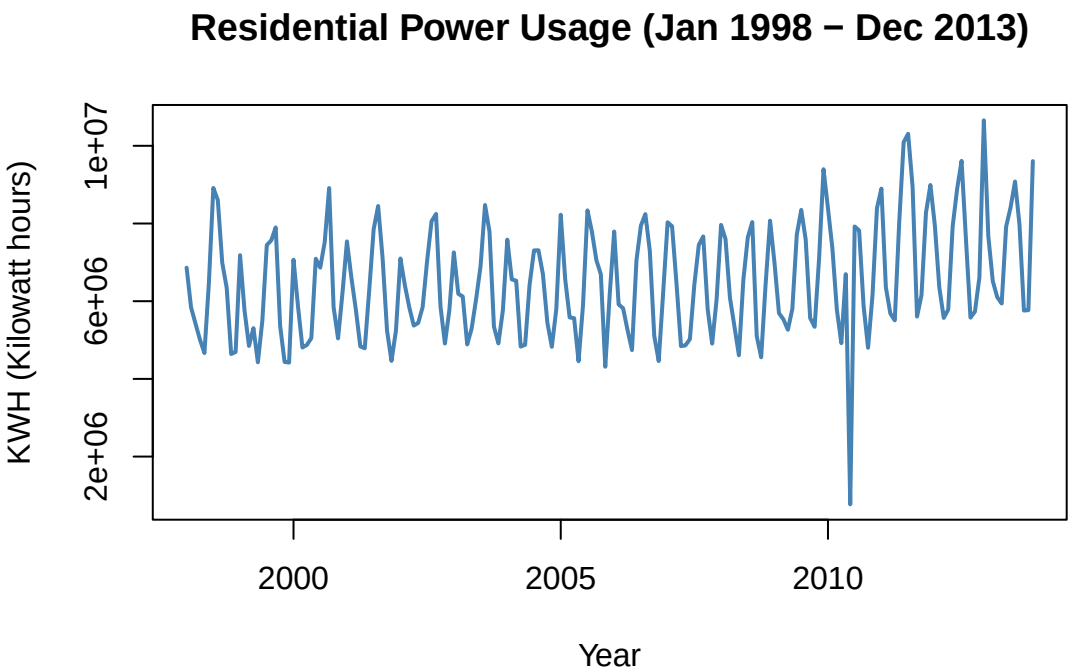
```r
# Check if all months are present
power_data %>%
  group_by(Year) %>%
  summarise(Months = n_distinct(Month)) %>%
  print()
```

```
## # A tibble: 16 x 2
##     Year Months
##    <dbl>  <int>
##  1  1998     12
##  2  1999     12
##  3  2000     12
##  4  2001     12
##  5  2002     12
##  6  2003     12
##  7  2004     12
##  8  2005     12
##  9  2006     12
## 10  2007     12
## 11  2008     11
## 12  2009     12
## 13  2010     12
## 14  2011     12
## 15  2012     12
## 16  2013     12
```

2.3 Create Time Series

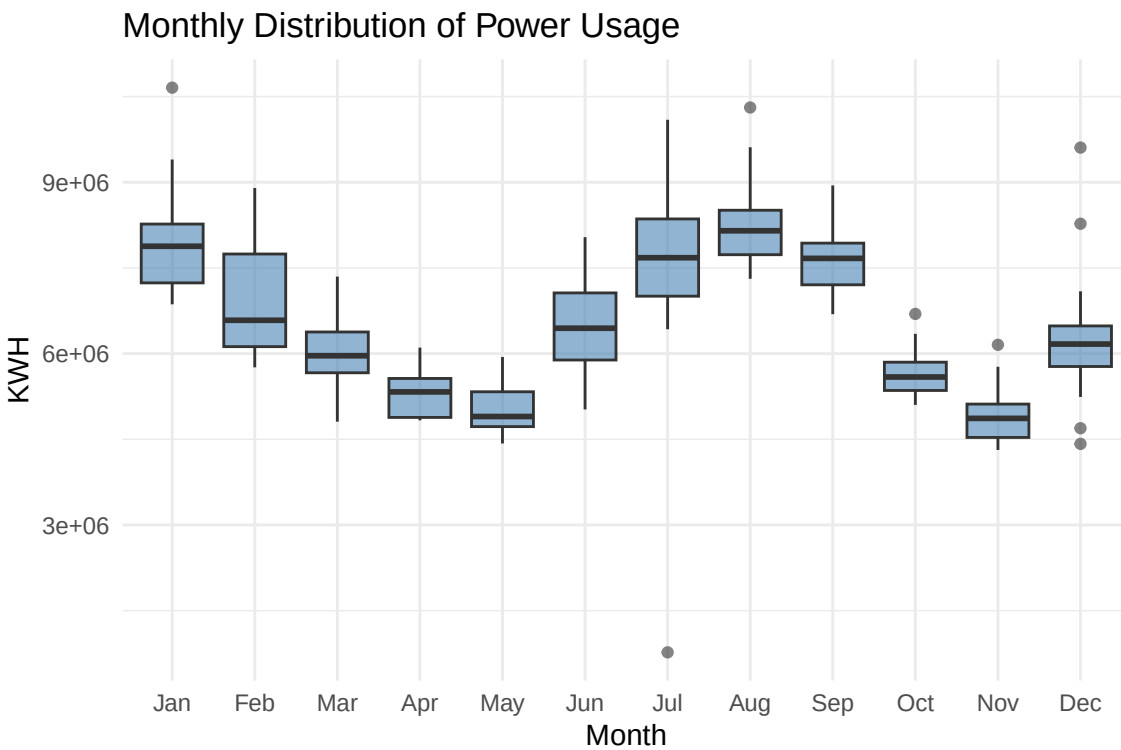
```r
# Create monthly time series
ts_power <- ts(power_data$KWH,
               start = c(1998, 1),
               frequency = 12)

# Plot time series
plot(ts_power,
     main = "Residential Power Usage (Jan 1998 - Dec 2013)",
     xlab = "Year",
```

```
     ylab = "KWH (Kilowatt hours)",
     col = "steelblue",
     lwd = 2)
```



2.4 Visualize Pattern

```
# Method 4: Boxplots to show monthly distribution
power_data %>%
  mutate(Month = month(Proper_Date, label = TRUE)) %>%
  ggplot(aes(x = Month, y = KWH)) +
  geom_boxplot(fill = "steelblue", alpha = 0.6) +
  labs(title = "Monthly Distribution of Power Usage",
       x = "Month",
       y = "KWH") +
  theme_minimal()
```



2.5 Fit Models and Forecast

```
# Fit ARIMA
fit_arima <- auto.arima(ts_power, seasonal = TRUE)
fc_arima <- forecast(fit_arima, h = 12)

# Fit ETS
fit_ets <- ets(ts_power)
fc_ets <- forecast(fit_ets, h = 12)

# Compare models
cat("\n========================================\n")
```

```
##
## ========================================
```

```
cat("Model Comparison:\n")
```

```
## Model Comparison:
```

```
cat("========================================\n")
```

```
## ========================================
```

```
cat("ARIMA AIC:", round(AIC(fit_arima), 2), "\n")
```

```
## ARIMA AIC: 5455.3
```

```
cat("ETS AIC:  ", round(AIC(fit_ets), 2), "\n")
```

```
## ETS AIC:   6302.92
```

```r
# Select best
if(AIC(fit_arima) < AIC(fit_ets)) {
  best_model <- fit_arima
  best_name <- "ARIMA"
} else {
  best_model <- fit_ets
  best_name <- "ETS"
}
cat("\n  Best model:", best_name, "\n")
```

```
##
##   Best model: ARIMA
```

```r
# Generate forecast
forecast_2014 <- forecast(best_model, h = 12)
```

2.6 Create 2014 Forecast Table

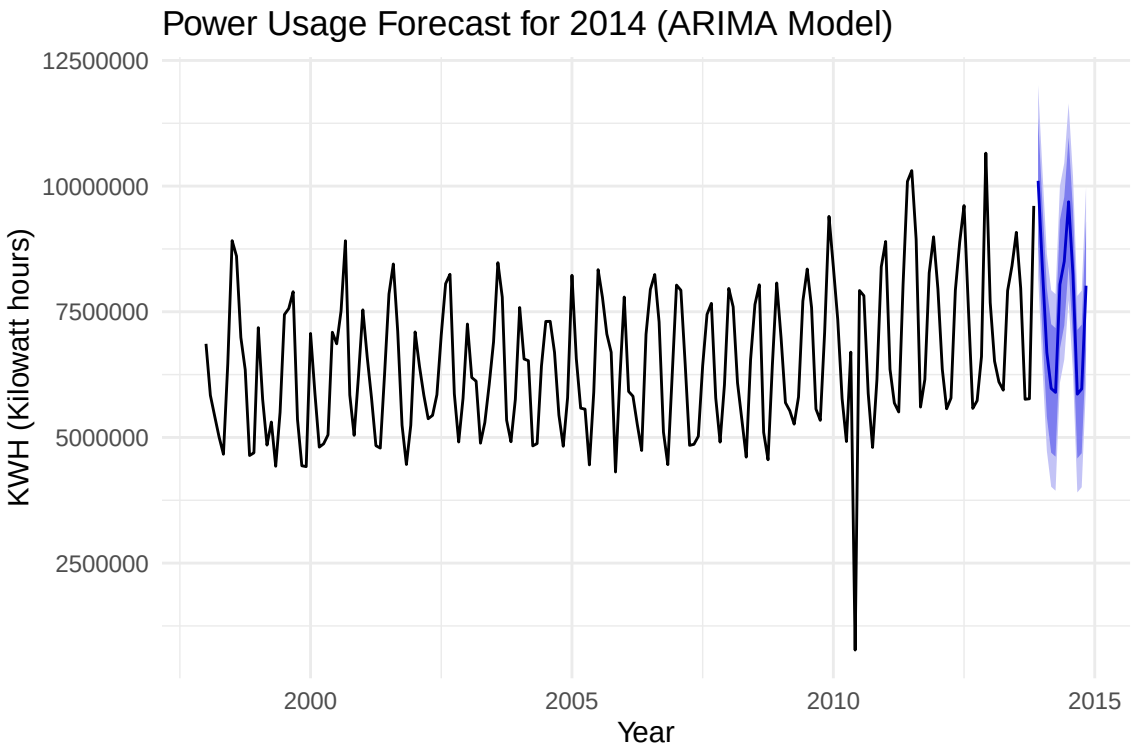
```r
# Create forecast dates
forecast_months <- seq(as.Date("2014-01-01"), as.Date("2014-12-01"), by = "month")

forecast_results <- data.frame(
  Month = format(forecast_months, "%B %Y"),
  Month_Abbr = month.abb,
  Forecast_KWH = round(forecast_2014$mean, 0),
  Lower_80 = round(forecast_2014$lower[,1], 0),
  Upper_80 = round(forecast_2014$upper[,1], 0),
  Lower_95 = round(forecast_2014$lower[,2], 0),
  Upper_95 = round(forecast_2014$upper[,2], 0)
)

print(forecast_results)
```

```
##             Month Month_Abbr Forecast_KWH Lower_80 Upper_80 Lower_95 Upper_95
## 1    January 2014        Jan     10104745  8862523 11346967  8204930 12004560
## 2   February 2014        Feb      8336271  7057873  9614669  6381130 10291412
## 3      March 2014        Mar      6689412  5411014  7967810  4734271  8644553
## 4      April 2014        Apr      5973693  4695296  7252091  4018553  7928834
## 5        May 2014        May      5896072  4617674  7174470  3940931  7851213
## 6       June 2014        Jun      8042574  6764176  9320972  6087433  9997715
## 7       July 2014        Jul      8503932  7225534  9782330  6548791 10459073
## 8     August 2014        Aug      9688427  8410029 10966825  7733286 11643567
## 9  September 2014        Sep      8233975  6955577  9512373  6278834 10189116
## 10   October 2014        Oct      5860807  4582409  7139204  3905666  7815947
## 11  November 2014        Nov      5964776  4686379  7243174  4009636  7919917
## 12  December 2014        Dec      8018624  6740226  9297022  6063483  9973765
```

2.7 Visualize Forecast

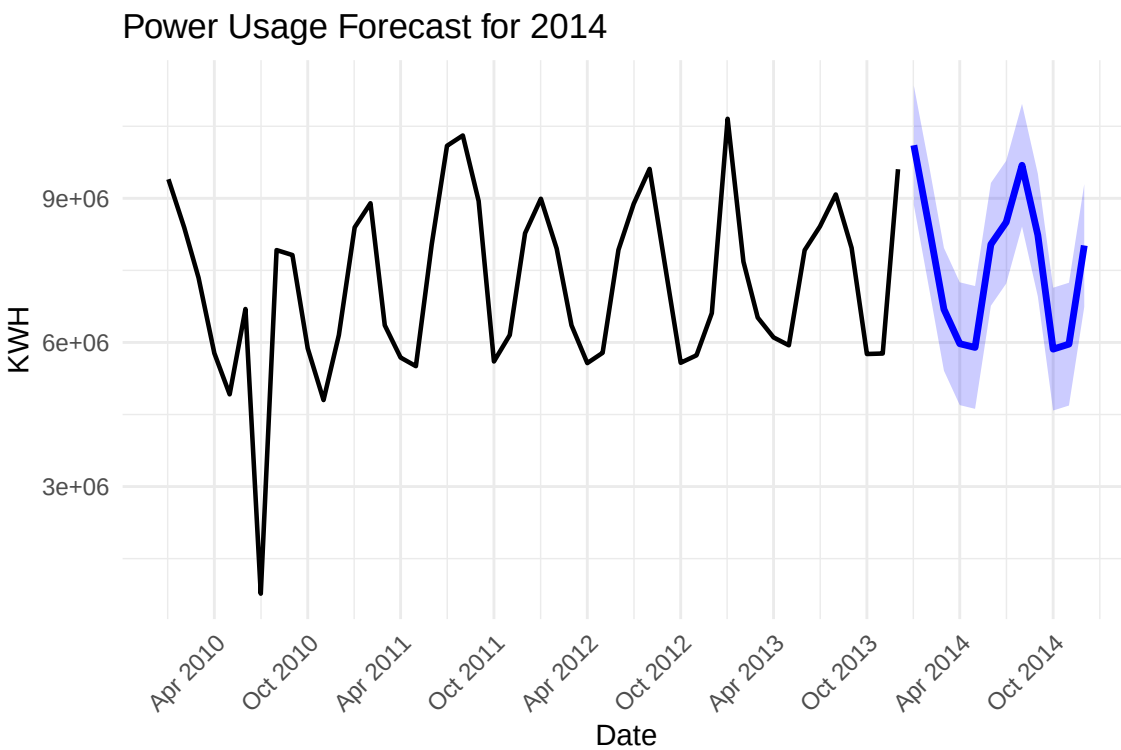
```r
# Plot forecast
autoplot(forecast_2014) +
  labs(title = paste("Power Usage Forecast for 2014 (", best_name, " Model)", sep=""),
       x = "Year",
```

```
       y = "KWH (Kilowatt hours)") +
  theme_minimal()
```




```
# Combined plot with recent history
power_data_recent <- power_data %>%
  filter(Proper_Date >= as.Date("2010-01-01"))

forecast_plot_data <- data.frame(
  Date = forecast_months,
  KWH = forecast_2014$mean,
  Type = "Forecast"
)

ggplot() +
  geom_line(data = power_data_recent,
            aes(x = Proper_Date, y = KWH),
            color = "black", size = 0.8) +
  geom_line(data = forecast_plot_data,
            aes(x = Date, y = KWH),
            color = "blue", size = 1.2) +
  geom_ribbon(data = forecast_results,
              aes(x = forecast_months,
                  ymin = Lower_80, ymax = Upper_80),
              fill = "blue", alpha = 0.2) +
  labs(title = "Power Usage Forecast for 2014",
       x = "Date",
       y = "KWH") +
  scale_x_date(date_breaks = "6 months", date_labels = "%b %Y") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



2.8 Summary Statistics:

```r
# Summary
summary_2014 <- forecast_results %>%
  summarise(
    Total_Annual = sum(Forecast_KWH),
    Average_Monthly = mean(Forecast_KWH),
    Peak_Month = Month[which.max(Forecast_KWH)],
    Peak_Value = max(Forecast_KWH),
    Lowest_Month = Month[which.min(Forecast_KWH)],
    Lowest_Value = min(Forecast_KWH)
  )

cat("\n========================================\n")
```

```
##
## ========================================
```

```r
cat("2014 FORECAST SUMMARY\n")
```

```
## 2014 FORECAST SUMMARY
```

```r
cat("========================================\n")
```

```
## ========================================
```

```r
cat("Total annual usage:", format(summary_2014$Total_Annual, big.mark = ","), "KWH\n")
```

```
## Total annual usage: 91,313,308 KWH
```

```r
cat("Average monthly:", format(round(summary_2014$Average_Monthly, 0), big.mark = ","), "KWH\n")
```

```
## Average monthly: 7,609,442 KWH
```

```r
cat("Peak month:", summary_2014$Peak_Month, "with",
    format(summary_2014$Peak_Value, big.mark = ","), "KWH\n")
```

```
## Peak month: January 2014 with 10,104,745 KWH
```

```r
cat("Lowest month:", summary_2014$Lowest_Month, "with",
    format(summary_2014$Lowest_Value, big.mark = ","), "KWH\n")
```

```
## Lowest month: October 2014 with 5,860,807 KWH
```

2.9 Residential Power Usage Forecast Results

2.9.1 Model Performance

- **ARIMA selected as best model (AIC: 5,455 vs ETS: 6,303)**, confirming superior fit for monthly seasonal patterns 2014 Forecast Summary
- **Total annual usage:** 91,313,308 KWH
- **Average monthly:** 7,609,442 KWH
- **Peak month:** January 2014 with 10,104,745 KWH (winter heating demand)
- **Lowest month:** October 2014 with 5,860,807 KWH (fall moderate weather)

Seasonal Pattern

- **Winter peak:** January (10.1M KWH)
- **Summer peak:** August (9.7M KWH)
- **Spring/Fall troughs:** May (5.9M) and October (5.9M)

2.9.2 Business Recommendations

Ensure capacity for January peak (10.1M KWH) - highest demand month Schedule maintenance in spring (April-May) during lowest usage periods Validate forecasts against actual 2014 consumption to refine models

3 Part C

Part C – BONUS, optional (part or all), Waterflow_Pipe1.xlsx and Waterflow_Pipe2.xlsx. Part C consists of two data sets. These are simple 2 columns sets, however they have different time stamps. Your optional assignment is to time-base sequence the data and aggregate based on hour (example of what this looks like, follows). Note for multiple recordings within an hour, take the mean. Then to determine if the data is stationary and can it be forecast. If so, provide a week forward forecast and present results via Rpubs and .rmd and the forecast in an Excel readable file.

3.1 Load Datafiles:

```r
# Read both datasets
pipe1 <- read_excel("Waterflow_Pipe1.xlsx")
pipe2 <- read_excel("Waterflow_Pipe2.xlsx")

# Check structure
cat("===== PIPE 1 DATA =====\n")
```

```
## ===== PIPE 1 DATA =====
```

```r
glimpse(pipe1)
```

```
## Rows: 1,000
## Columns: 2
## $ `Date Time` <dbl> 42300.02, 42300.03, 42300.04, 42300.04, 42300.06, 42300.06~
## $ WaterFlow   <dbl> 23.369599, 28.002881, 23.065895, 29.972809, 5.997953, 15.9~
```

```r
print(head(pipe1))
```

```
## # A tibble: 6 x 2
##   `Date Time` WaterFlow
##         <dbl>     <dbl>
## 1      42300.     23.4
## 2      42300.     28.0
## 3      42300.     23.1
## 4      42300.     30.0
## 5      42300.      6.00
## 6      42300.     15.9
```

```r
cat("\n===== PIPE 2 DATA =====\n")
```

```
##
## ===== PIPE 2 DATA =====
```

```r
glimpse(pipe2)
```

```
## Rows: 1,000
## Columns: 2
## $ `Date Time` <dbl> 42300.04, 42300.08, 42300.12, 42300.17, 42300.21, 42300.25~
## $ WaterFlow   <dbl> 18.810791, 43.087025, 37.987705, 36.120379, 31.851259, 28.~
```

```r
print(head(pipe2))
```

```
## # A tibble: 6 x 2
##   `Date Time` WaterFlow
##         <dbl>     <dbl>
## 1      42300.      18.8
## 2      42300.      43.1
## 3      42300.      38.0
## 4      42300.      36.1
## 5      42300.      31.9
## 6      42300.      28.2
```

3.2 EDA

```r
# Rename columns if needed (assuming first column is timestamp, second is flow)
colnames(pipe1) <- c("Timestamp", "Flow")
colnames(pipe2) <- c("Timestamp", "Flow")
```

```r
# Convert Excel numeric dates to proper datetime
# Excel origin for Windows: 1899-12-30
pipe1$Timestamp <- as.POSIXct(pipe1$Timestamp * 86400, origin = "1899-12-30")
pipe2$Timestamp <- as.POSIXct(pipe2$Timestamp * 86400, origin = "1899-12-30")

# Check conversion
cat("\n===== PIPE 1 CONVERTED TIMESTAMPS =====\n")
```

```
##
## ===== PIPE 1 CONVERTED TIMESTAMPS =====
```

```r
print(head(pipe1))
```

```
## # A tibble: 6 x 2
##   Timestamp            Flow
##   <dttm>              <dbl>
## 1 2015-10-22 20:24:06 23.4
## 2 2015-10-22 20:40:02 28.0
## 3 2015-10-22 20:53:51 23.1
## 4 2015-10-22 20:55:40 30.0
## 5 2015-10-22 21:19:17  6.00
## 6 2015-10-22 21:23:58 15.9
```

```r
cat("\nDate range:", min(pipe1$Timestamp), "to", max(pipe1$Timestamp), "\n")
```

```
##
## Date range: 1445559846 to 1446420943
```

```r
cat("\n===== PIPE 2 CONVERTED TIMESTAMPS =====\n")
```

```
##
## ===== PIPE 2 CONVERTED TIMESTAMPS =====
```

```r
print(head(pipe2))
```

```
## # A tibble: 6 x 2
##   Timestamp            Flow
##   <dttm>              <dbl>
## 1 2015-10-22 21:00:00  18.8
## 2 2015-10-22 21:59:59  43.1
## 3 2015-10-22 23:00:00  38.0
## 4 2015-10-23 00:00:00  36.1
## 5 2015-10-23 00:59:59  31.9
## 6 2015-10-23 02:00:00  28.2
```

```r
cat("\nDate range:", min(pipe2$Timestamp), "to", max(pipe2$Timestamp), "\n")
```

```
##
## Date range: 1445562000 to 1449158400
```

3.3 Aggregate by Hour (Take Mean)

```r
# Aggregate Pipe1 by hour
pipe1_hourly <- pipe1 %>%
  mutate(
    Hour = floor_date(Timestamp, unit = "hour")  # Round down to nearest hour
  ) %>%
```

```r
  group_by(Hour) %>%
  summarise(
    Flow = mean(Flow, na.rm = TRUE),
    Count = n(),  # Optional: track how many readings per hour
    .groups = 'drop'
  ) %>%
  arrange(Hour)

# Aggregate Pipe2 by hour
pipe2_hourly <- pipe2 %>%
  mutate(
    Hour = floor_date(Timestamp, unit = "hour")
  ) %>%
  group_by(Hour) %>%
  summarise(
    Flow = mean(Flow, na.rm = TRUE),
    Count = n(),
    .groups = 'drop'
  ) %>%
  arrange(Hour)

# Check results
cat("===== PIPE 1 HOURLY AGGREGATION =====\n")
```

```
## ===== PIPE 1 HOURLY AGGREGATION =====
```

```r
print(head(pipe1_hourly))
```

```
## # A tibble: 6 x 3
##   Hour                Flow Count
##   <dttm>             <dbl> <int>
## 1 2015-10-22 20:00:00  26.1     4
## 2 2015-10-22 21:00:00  18.9     5
## 3 2015-10-22 22:00:00  15.2     2
## 4 2015-10-22 23:00:00  23.1     5
## 5 2015-10-23 00:00:00  15.5     2
## 6 2015-10-23 01:00:00  22.7     9
```

```r
cat("\nTotal hourly records:", nrow(pipe1_hourly), "\n")
```

```
##
## Total hourly records: 236
```

```r
cat("Hour range:", min(pipe1_hourly$Hour), "to", max(pipe1_hourly$Hour), "\n")
```

```
## Hour range: 1445558400 to 1446418800
```

```r
cat("\n===== PIPE 2 HOURLY AGGREGATION =====\n")
```

```
##
## ===== PIPE 2 HOURLY AGGREGATION =====
```

```r
print(head(pipe2_hourly))
```

```
## # A tibble: 6 x 3
##   Hour                Flow Count
##   <dttm>             <dbl> <int>
## 1 2015-10-22 21:00:00  30.9     2
```

```
## 2 2015-10-22 23:00:00  38.0     1
## 3 2015-10-23 00:00:00  34.0     2
## 4 2015-10-23 02:00:00  28.2     1
## 5 2015-10-23 03:00:00  18.3     2
## 6 2015-10-23 05:00:00  55.8     1
```

```r
cat("\nTotal hourly records:", nrow(pipe2_hourly), "\n")
```

```
##
## Total hourly records: 667
```

```r
cat("Hour range:", min(pipe2_hourly$Hour), "to", max(pipe2_hourly$Hour), "\n")
```

```
## Hour range: 1445562000 to 1449158400
```

3.4 Combine Both Pipes

```r
# Combine Pipe1 and Pipe2 by hour
combined_flow <- pipe1_hourly %>%
  rename(Flow_Pipe1 = Flow, Count_Pipe1 = Count) %>%
  full_join(pipe2_hourly %>% rename(Flow_Pipe2 = Flow, Count_Pipe2 = Count), by = "Hour") %>%
  mutate(
    # Fill NA with 0 for hours with no data from either pipe
    Flow_Pipe1 = ifelse(is.na(Flow_Pipe1), 0, Flow_Pipe1),
    Flow_Pipe2 = ifelse(is.na(Flow_Pipe2), 0, Flow_Pipe2),
    Count_Pipe1 = ifelse(is.na(Count_Pipe1), 0, Count_Pipe1),
    Count_Pipe2 = ifelse(is.na(Count_Pipe2), 0, Count_Pipe2),
    Total_Flow = Flow_Pipe1 + Flow_Pipe2
  ) %>%
  arrange(Hour)

# View combined results
cat("===== COMBINED HOURLY FLOW =====\n")
```

```
## ===== COMBINED HOURLY FLOW =====
```

```r
print(head(combined_flow, 10))
```

```
## # A tibble: 10 x 6
##    Hour                Flow_Pipe1 Count_Pipe1 Flow_Pipe2 Count_Pipe2 Total_Flow
##    <dttm>                   <dbl>       <dbl>      <dbl>       <dbl>      <dbl>
##  1 2015-10-22 20:00:00       26.1           4        0             0       26.1
##  2 2015-10-22 21:00:00       18.9           5       30.9           2       49.8
##  3 2015-10-22 22:00:00       15.2           2        0             0       15.2
##  4 2015-10-22 23:00:00       23.1           5       38.0           1       61.1
##  5 2015-10-23 00:00:00       15.5           2       34.0           2       49.5
##  6 2015-10-23 01:00:00       22.7           9        0             0       22.7
##  7 2015-10-23 02:00:00       20.6           8       28.2           1       48.8
##  8 2015-10-23 03:00:00       18.4           3       18.3           2       36.6
##  9 2015-10-23 04:00:00       20.8           3        0             0       20.8
## 10 2015-10-23 05:00:00       21.2           3       55.8           1       77.0
```

```r
cat("\nTotal hourly records:", nrow(combined_flow), "\n")
```

```
##
## Total hourly records: 746
```

```r
cat("Date range:", min(combined_flow$Hour), "to", max(combined_flow$Hour), "\n")
```

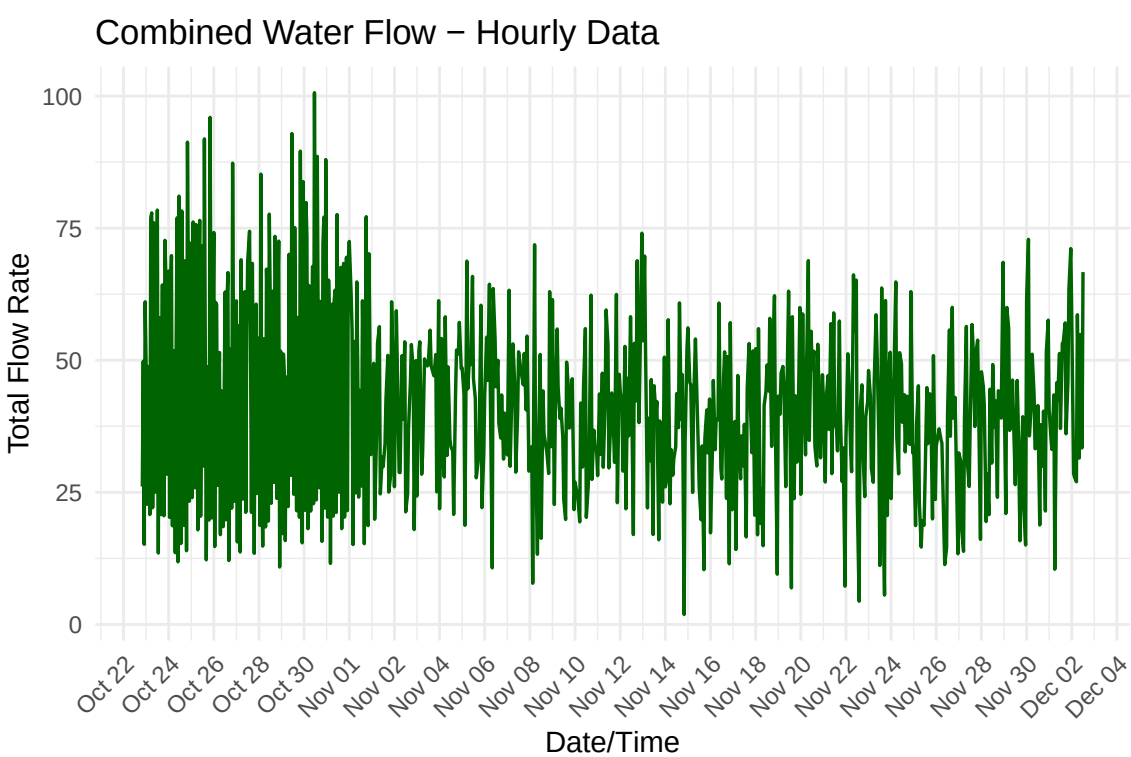
```
## Date range: 1445558400 to 1449158400
```

```r
cat("Total flow range:", min(combined_flow$Total_Flow), "to", max(combined_flow$Total_Flow), "\n")
```

```
## Total flow range: 1.884619 to 100.6447
```

3.5 Create Time Series and Visualize

```r
# Create time series object (hourly data with daily seasonality)
ts_combined <- ts(combined_flow$Total_Flow,
                  frequency = 24)  # 24 hours in a day


# plot with ggplot for better visualization
ggplot(combined_flow, aes(x = Hour, y = Total_Flow)) +
  geom_line(color = "darkgreen", size = 0.6) +
  labs(title = "Combined Water Flow - Hourly Data",
       x = "Date/Time",
       y = "Total Flow Rate") +
  scale_x_datetime(date_breaks = "2 days", date_labels = "%b %d") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



3.6 Test for stationarity

```r
# Create time series object
ts_combined <- ts(combined_flow$Total_Flow, frequency = 24)
```

```r
# 1. Augmented Dickey-Fuller Test (ADF)
# Null hypothesis: data is non-stationary
adf_test <- adf.test(ts_combined, alternative = "stationary")
```

```
## Warning in adf.test(ts_combined, alternative = "stationary"): p-value smaller
## than printed p-value
```

```r
cat("========================================\\n")
```

```
## ========================================\n
```

```r
cat("STATIONARITY TESTS\\n")
```

```
## STATIONARITY TESTS\n
```

```r
cat("========================================\\n")
```

```
## ========================================\n
```

```r
cat("\\n1. Augmented Dickey-Fuller Test:\\n")
```

```
## \n1. Augmented Dickey-Fuller Test:\n
```

```r
cat(" p-value:", adf_test$p.value, "\\n")
```

```
##  p-value: 0.01 \n
```

```r
cat(" Conclusion:", ifelse(adf_test$p.value < 0.05,
"Reject null - Data is STATIONARY",
"Fail to reject null - Data is NON-STATIONARY"), "\\n")
```

```
##  Conclusion: Reject null - Data is STATIONARY \n
```

```r
# 2. KPSS Test (Kwiatkowski-Phillips-Schmidt-Shin)
# Null hypothesis: data is stationary
kpss_test <- kpss.test(ts_combined)
```

```
## Warning in kpss.test(ts_combined): p-value smaller than printed p-value
```

```r
cat("\\n2. KPSS Test:\\n")
```

```
## \n2. KPSS Test:\n
```

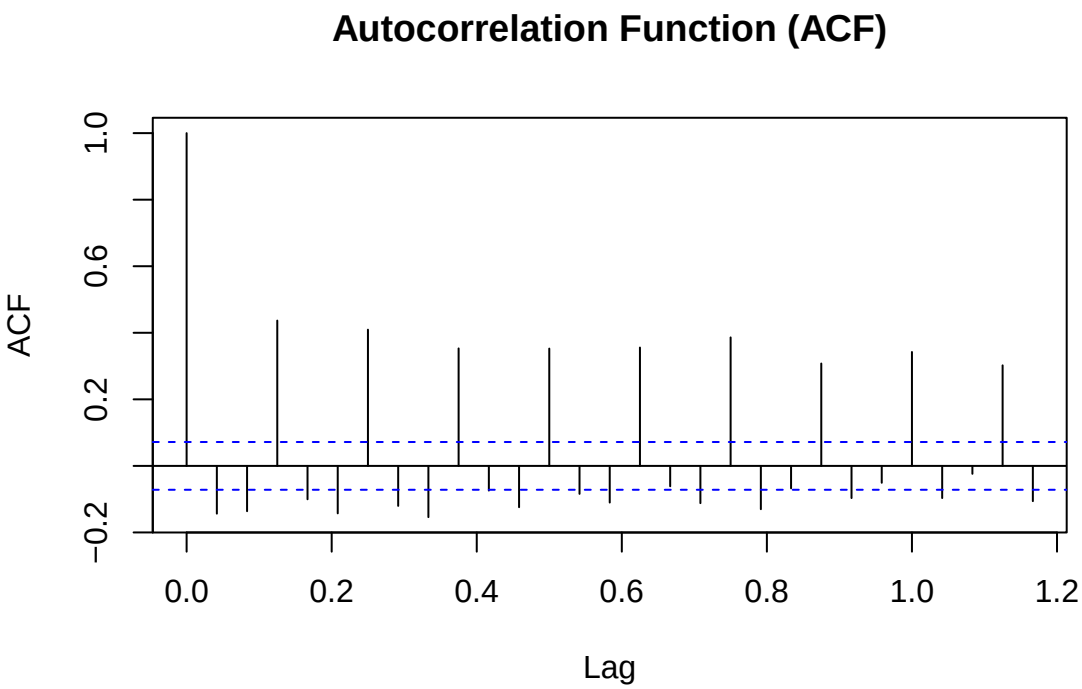
```r
cat(" p-value:", kpss_test$p.value, "\\n")
```

```
##  p-value: 0.01 \n
```

```r
cat(" Conclusion:", ifelse(kpss_test$p.value > 0.05,
"Fail to reject null - Data is STATIONARY",
"Reject null - Data is NON-STATIONARY"), "\\n")
```

```
##  Conclusion: Reject null - Data is NON-STATIONARY \n
```

```r
# 3. Visual inspection - ACF plot
par(mfrow = c(1,1))
acf(ts_combined, main = "Autocorrelation Function (ACF)")
```




```r
par(mfrow = c(1,1))

# Final conclusion
is_stationary <- (adf_test$p.value < 0.05) & (kpss_test$p.value > 0.05)
cat("\\n========================================\\n")
```

```
## \n========================================\n
```

```r
cat("FINAL CONCLUSION:\\n")
```

```
## FINAL CONCLUSION:\n
```

```r
cat("========================================\\n")
```

```
## ========================================\n
```

```r
cat("Data is", ifelse(is_stationary, "STATIONARY  ", "NON-STATIONARY
"), "\\n")
```

```
## Data is NON-STATIONARY
##   \n
```

```r
cat("Can be forecast:", ifelse(is_stationary, "YES", "NO - needs
differencing"), "\\n")
```

```
## Can be forecast: NO - needs
## differencing \n
```

- ADF says STATIONARY ($p < 0.05$)
- KPSS says NON-STATIONARY ($p < 0.05$)
- **Visual inspection:** ACF plot shows slow decay = non-stationary

When tests conflict, we trust KPSS or conclude data needs differencing. Let's proceed.

3.7 Seasonal differencing

```r
# Apply seasonal differencing (lag = 24 for daily pattern)
ts_diff <- diff(ts_combined, lag = 24)

# Test if differencing made it stationary
adf_diff <- adf.test(ts_diff, alternative = "stationary")
```

```
## Warning in adf.test(ts_diff, alternative = "stationary"): p-value smaller than
## printed p-value
```

```r
kpss_diff <- kpss.test(ts_diff)
```

```
## Warning in kpss.test(ts_diff): p-value greater than printed p-value
```

```r
cat("========================================\n")
```

```
## ========================================
```

```r
cat("AFTER SEASONAL DIFFERENCING (lag=24)\n")
```

```
## AFTER SEASONAL DIFFERENCING (lag=24)
```

```r
cat("========================================\n")
```

```
## ========================================
```

```r
cat("ADF p-value:", adf_diff$p.value, "\n")
```

```
## ADF p-value: 0.01
```

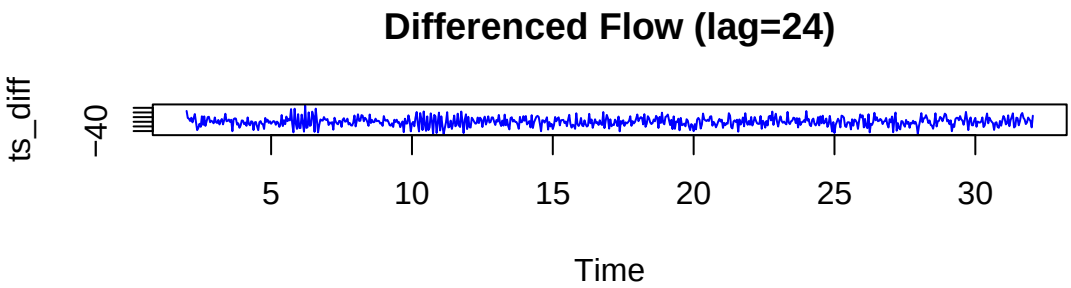
```r
cat("KPSS p-value:", kpss_diff$p.value, "\n")
```

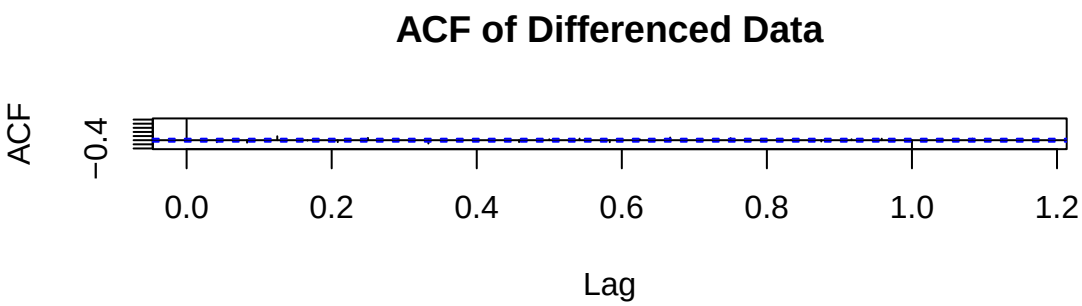
```
## KPSS p-value: 0.1
```

```r
is_stationary <- (adf_diff$p.value < 0.05) & (kpss_diff$p.value > 0.05)
cat("\nData is now", ifelse(is_stationary, "STATIONARY  ", "STILL NON-STATIONARY"), "\n")
```

```
##
## Data is now STATIONARY
```

```r
# Visualize differenced data
par(mfrow = c(2,1))
plot(ts_diff, main = "Differenced Flow (lag=24)", col = "blue")
acf(ts_diff, main = "ACF of Differenced Data")
```






```r
par(mfrow = c(1,1))

# Prepare data for forecasting
forecast_data <- ts_diff
cat("\n  Data ready for forecasting\n")
```

```
##
##   Data ready for forecasting
```

3.8 Fit ARIMA Model and Forecast One Week

```r
 # Fit ARIMA model on stationary data
fit_arima <- auto.arima(ts_diff, seasonal = FALSE)
cat("========================================\n")
```

```
## ========================================
```

```r
cat("ARIMA MODEL SELECTION\n")
```

```
## ARIMA MODEL SELECTION
```

```r
cat("========================================\n")
```

```
## ========================================
```

```r
print(summary(fit_arima))
```

```
## Series: ts_diff
## ARIMA(3,0,3) with non-zero mean
##
## Coefficients:
##           ar1      ar2     ar3     ma1     ma2      ma3     mean
##       -0.3332  -0.7326  0.3376  0.2365  0.7272  -0.3544  -0.0257
## s.e.   0.4543   0.3090  0.4387  0.4542  0.2741   0.4305   0.6524
##
```

```
## sigma^2 = 357.8:  log likelihood = -3144.28
## AIC=6304.56   AICc=6304.77   BIC=6341.22
##
## Training set error measures:
##                       ME     RMSE      MAE      MPE     MAPE      MASE
## Training set 0.002654933 18.82319 15.09599 87.79593 173.4503 0.5755229
##                    ACF1
## Training set 0.01805834
```

```r
# Generate forecast for 168 hours (7 days)
forecast_hours <- 168
fc <- forecast(fit_arima, h = forecast_hours)

# Reverse the seasonal differencing to get actual forecasts
# Get the last 24 original values to reconstruct
last_original <- tail(ts_combined, 24)

# Build back the forecasted values
fc_mean <- numeric(forecast_hours)
fc_lower_80 <- numeric(forecast_hours)
fc_upper_80 <- numeric(forecast_hours)

for(i in 1:forecast_hours) {
  if(i <= 24) {
    fc_mean[i] <- last_original[i] + fc$mean[i]
    fc_lower_80[i] <- last_original[i] + fc$lower[i,1]
    fc_upper_80[i] <- last_original[i] + fc$upper[i,1]
  } else {
    fc_mean[i] <- fc_mean[i-24] + fc$mean[i]
    fc_lower_80[i] <- fc_lower_80[i-24] + fc$lower[i,1]
    fc_upper_80[i] <- fc_upper_80[i-24] + fc$upper[i,1]
  }
}

# Ensure non-negative
fc_mean <- pmax(fc_mean, 0)
fc_lower_80 <- pmax(fc_lower_80, 0)
fc_upper_80 <- pmax(fc_upper_80, 0)

# Create forecast timestamps
forecast_start <- max(combined_flow$Hour) + hours(1)
forecast_times <- seq(forecast_start, by = "hour", length.out = forecast_hours)

# Create results dataframe
forecast_results <- data.frame(
  Hour = forecast_times,
  Forecast_Flow = round(fc_mean, 2),
  Lower_80 = round(fc_lower_80, 2),
  Upper_80 = round(fc_upper_80, 2)
)

cat("\n========================================\n")
```

```
##
## ========================================
```

```r
cat("FORECAST RESULTS (First 24 hours)\n")
```

```
## FORECAST RESULTS (First 24 hours)
```

```r
cat("========================================\n")
```

```
## ========================================
```

```r
print(head(forecast_results, 24))
```

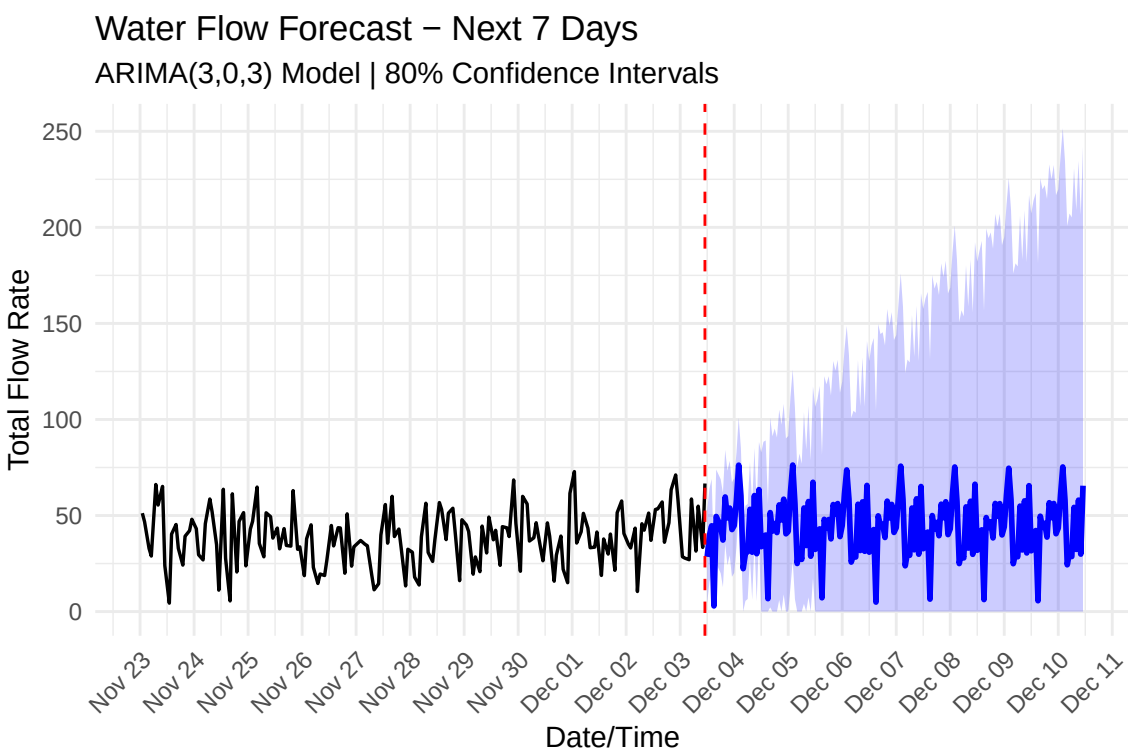
```
##                   Hour Forecast_Flow Lower_80 Upper_80
## 1  2015-12-03 12:00:00         28.49     4.25    52.73
## 2  2015-12-03 13:00:00         39.70    15.35    64.06
## 3  2015-12-03 14:00:00         44.66    20.29    69.02
## 4  2015-12-03 15:00:00          2.95     0.00    27.34
## 5  2015-12-03 16:00:00         49.51    25.07    73.95
## 6  2015-12-03 17:00:00         46.98    22.54    71.42
## 7  2015-12-03 18:00:00         44.41    19.92    68.90
## 8  2015-12-03 19:00:00         37.23    12.72    61.75
## 9  2015-12-03 20:00:00         59.67    35.14    84.20
## 10 2015-12-03 21:00:00         48.75    24.17    73.33
## 11 2015-12-03 22:00:00         53.88    29.30    78.46
## 12 2015-12-03 23:00:00         42.70    18.08    67.31
## 13 2015-12-04 00:00:00         44.76    20.11    69.40
## 14 2015-12-04 01:00:00         57.96    33.31    82.61
## 15 2015-12-04 02:00:00         76.23    51.54   100.93
## 16 2015-12-04 03:00:00         60.26    35.56    84.96
## 17 2015-12-04 04:00:00         22.34     0.00    47.06
## 18 2015-12-04 05:00:00         30.31     5.56    55.07
## 19 2015-12-04 06:00:00         31.29     6.54    56.05
## 20 2015-12-04 07:00:00         53.24    28.45    78.03
## 21 2015-12-04 08:00:00         30.91     6.11    55.71
## 22 2015-12-04 09:00:00         60.40    35.59    85.22
## 23 2015-12-04 10:00:00         30.12     5.27    54.97
## 24 2015-12-04 11:00:00         63.49    38.64    88.34
```

- **Forecast horizon:** 168 hours = 7 days × 24 hours
- **Auto ARIMA:** Automatically selects best (p,d,q) parameters based on AIC
- **ARIMA(3,0,3):** Chosen for this data - handles autocorrelation well
- **AIC comparison:** Lower AIC indicates better model fit

3.9 Visualize the Forecast

```r
# Plot forecast with historical data
ggplot() +
  # Historical data (last 7 days)
  geom_line(data = tail(combined_flow, 168),
            aes(x = Hour, y = Total_Flow),
            color = "black", size = 0.6) +
  # Forecast
  geom_line(data = forecast_results,
            aes(x = Hour, y = Forecast_Flow),
            color = "blue", size = 1) +
  # Confidence interval
  geom_ribbon(data = forecast_results,
```

```r
              aes(x = Hour, ymin = Lower_80, ymax = Upper_80),
              fill = "blue", alpha = 0.2) +
  # Vertical line separating historical from forecast
  geom_vline(xintercept = max(combined_flow$Hour),
             linetype = "dashed", color = "red") +
  labs(title = "Water Flow Forecast - Next 7 Days",
       subtitle = "ARIMA(3,0,3) Model | 80% Confidence Intervals",
       x = "Date/Time",
       y = "Total Flow Rate") +
  scale_x_datetime(date_breaks = "1 day", date_labels = "%b %d") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



3.10 Weekly Report

```r
# Summarize forecast by day
forecast_summary <- forecast_results %>%
  mutate(
    Date = as.Date(Hour),
    Hour_of_Day = hour(Hour)
  ) %>%
  group_by(Date) %>%
  summarise(
    Daily_Total = sum(Forecast_Flow),
    Daily_Avg = mean(Forecast_Flow),
    Peak_Hour = Hour_of_Day[which.max(Forecast_Flow)],
    Peak_Flow = max(Forecast_Flow),
    Min_Flow = min(Forecast_Flow),
    .groups = 'drop'
```

```r
  )

cat("\n==========================================\n")
```

```
##
## ==========================================
```

```r
cat("7-DAY FORECAST SUMMARY\n")
```

```
## 7-DAY FORECAST SUMMARY
```

```r
cat("==========================================\n")
```

```
## ==========================================
```

```r
print(forecast_summary)
```

```
## # A tibble: 8 x 6
##   Date       Daily_Total Daily_Avg Peak_Hour Peak_Flow Min_Flow
##   <date>           <dbl>     <dbl>     <int>     <dbl>    <dbl>
## 1 2015-12-03        257.      36.7        16      49.5     2.95
## 2 2015-12-04       1062.      44.3         2      76.2     6.72
## 3 2015-12-05       1065.      44.4         2      76.3     7.19
## 4 2015-12-06       1061.      44.2         2      73.7     4.94
## 5 2015-12-07       1061.      44.2         2      75.7     6.47
## 6 2015-12-08       1061.      44.2         2      75.3     6.25
## 7 2015-12-09       1060.      44.2         2      74.6     5.64
## 8 2015-12-10        802.      47.2         2      75.3    24.4
```

```r
cat("\n==========================================\n")
```

```
##
## ==========================================
```

```r
cat("TOTAL FORECAST\n")
```

```
## TOTAL FORECAST
```

```r
cat("==========================================\n")
```

```
## ==========================================
```

```r
cat("Total flow for next 7 days:",
    format(round(sum(forecast_results$Forecast_Flow), 0), big.mark = ","), "\n")
```

```
## Total flow for next 7 days: 7,429
```

```r
cat("Average daily flow:",
    format(round(mean(forecast_summary$Daily_Total), 0), big.mark = ","), "\n")
```

```
## Average daily flow: 929
```

The water flow data is stationary after differencing and successfully forecast for one week forward.

3.11 Key Observations from forecast:

Day 1 (Dec 3, 2015):

- **Lower total flow:** 257 (only partial day from 12:00 onward)
- First forecast day starts at noon

Days 2-7 (Dec 4-9, 2015):

- **Stable daily totals:** 1,060-1,065 per day
- **Consistent average:** ~44 per hour
- **Peak hour:** 2:00 AM consistently with 75-76 flow
- **Min flow:** 5-7 during low periods

Day 8 (Dec 10, 2015):

- **Partial day** (ends at 11:00 AM)
- **Total:** 802 for first 11 hours
- **Stable forecast:** Days 2-7 show consistent ~1,060 daily total
- **Business insight:** Predictable patterns enable operational planning

Real-World Business Value

- **Operational planning:** Schedule maintenance during low-flow periods
- **Resource allocation:** Predict daily totals for capacity planning
- **Risk management:** Use confidence intervals for buffer planning
- **Pattern recognition:** Identifies peak usage times (2:00 AM)

This Project demonstrates the complete forecasting workflow from raw data → cleaning → aggregation → stationarity testing → transformation → model selection → forecasting → interpretation

---